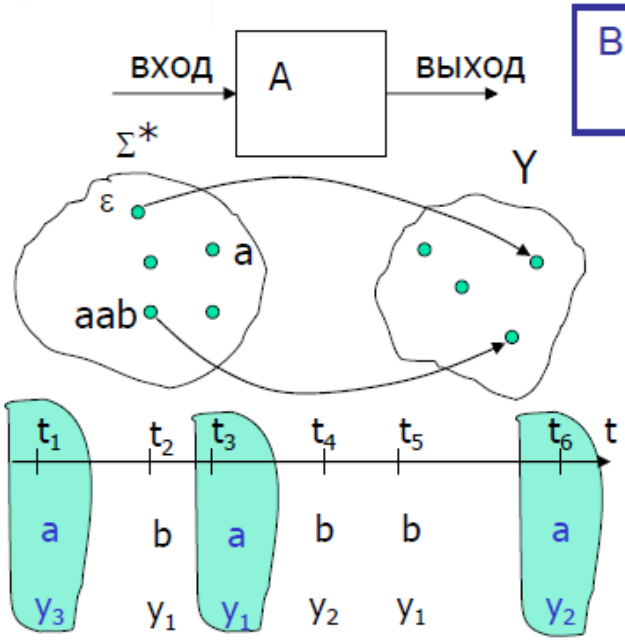
Теория алгоритмов

Дискретные устройства с памятью – конечные автоматы

Преобразователи информации с памятью



Выход зависит не только от входа, но и от истории

История – цепочка входных воздействий

Σ = {a, b}; это входные сигналы

Σ^* = {ε, a, b, aa, ab, bb, ..., baaba,...} это - истории

Каждый новый входной сигнал вызывает не только выход, но и изменение истории

Отображение $\Sigma \rightarrow Y$ не функциональное!
(одинаковым аргументам – разные выходы)

Реакция y на один и тот же вход – разная (выход зависит от состояния ПАМЯТИ)

Отображение $\Sigma^* \rightarrow Y$ функциональное
(одинаковым аргументам (историям) – одинаковые выходы)

Как построить теорию преобразователей информации с памятью?

Формальной моделью преобразователей информации с памятью являются КОНЕЧНЫЕ АВТОМАТЫ

Понятие состояния – основа моделей динамических устройств с памятью

- Динамическая система – такая, которая изменяется со временем
- Состояние – характеристика s динамической системы в момент времени $t \in T$, такая, что s(t) полностью определяется эволюция (поведение) системы в будущем, т.е. все состояния s(t') для $t' > t$
- Такое свойство называется "марковым". В таких динамических системах с марковским свойством будущее поведение системы зависит только от текущего состояния и от внешних воздействий, и НЕ зависит от того, каким образом система пришла в это состояние (при фиксированном настоящем будущее не зависит от прошлого)
- Например, при прерывании программы ее состояние запоминается в стеке. Что такое состояние программы? Это значения всех переменных и регистров, в частности, регистра адреса текущей команды. Это вся информация, которой достаточно, чтобы при выходе из прерывания снова запустить программу. Не зависит от того, как вела себя программа раньше!!
- Большинство реальных процессов управления характеризуются дискретностью моментов выработки воздействий
- Простейшей моделью дискретных систем с конечной памятью, имеющей огромное число применений, является КОНЕЧНЫЙ АВТОМАТ



Математические абстракции, а не физические объекты

Вертикальный фрезерный станок-автомат



Автомат по продаже пива



Системы управления этими устройствами строятся на основе теории автоматов

- Мы будем изучать автоматы – не физические объекты, а математические абстракции, абстрактные модели устройств-преобразователей информации с с памятью, которые выполняют свои функции в соответствии с предопределенным набором правил. Эти абстракции позволяют построить системы управления автоматическими устройствами
- Автомат в информатике в общем случае нельзя считать устройством, которое “выполняет” некоторую “работу”, это формализм, модель
- Абстрактные модели конечных автоматов-преобразователей, изучаемые нами в этом курсе, всегда можно реализовать. Обычно это устройства, управляющие чем-либо (не автомат Калашникова)

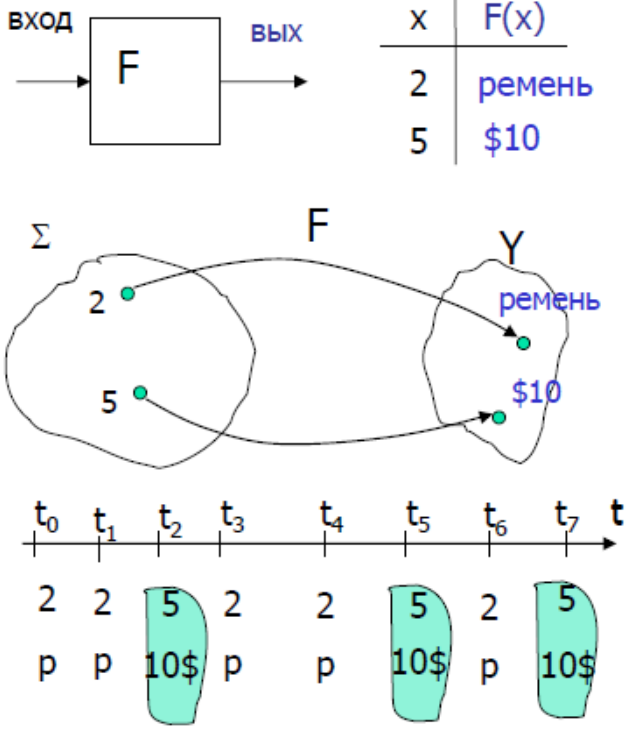
Теория автоматов

- Теория автоматов изучает абстрактные машины - преобразователи информации с памятью (а не функциональные преобразователи)
- Ценность теории состоит в том, что это, с одной стороны, абстрактная, формальная математическая модель, позволяющая решать многие теоретические проблемы, в частности, в областях:
  - теория алгоритмов
  - теория формальных языков
- Многие проблемы разрешимы, для реализации и анализа модели нужно конечное число ресурсов
- С другой стороны, у этой теории огромное число практических приложений: модель КА (хотя и не всегда) можно реализовать аппаратно или программно, и тогда это будет просто дискретное техническое устройство или алгоритм для обработки и преобразования информации
- Примеры применений:
  - системы логического управления, драйверы устройств
  - протоколы коммуникации, реагирующие системы (reactive systems)
  - алгоритмы синтаксического анализа и трансляции языков программирования
  - спецификация поведения интеллектуальных агентов и биологических систем



Пример: проблема воспитания школьника

Модель грубого воспитания - функция



Модель "умного" воспитания – автоматная

Умное воспитание: помним историю. Но не всю!

Сын принес 2 – не пороть его сразу:

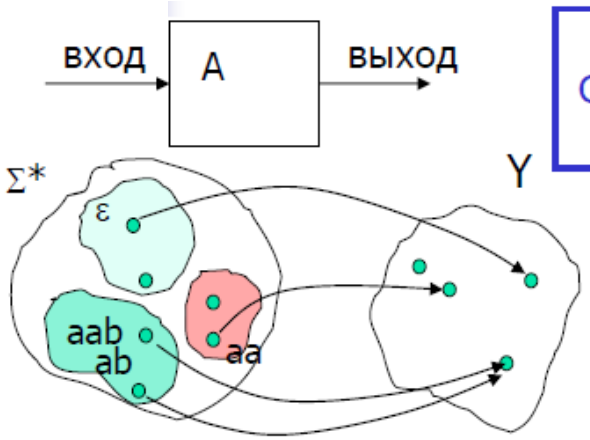
- если это случайная двойка, то успокоить;
- если две подряд - беспокоиться, порицать;
- уж если несколько двоек подряд, то ремень!

Сын принес 5 – не награждать его сразу:

- если это случайная пятерка, то порадоваться;
- если две подряд – хвалить: "Ведь можешь!";
- если уже несколько 5, то одаривать 10$

"Умное" воспитание состоит в том, что реакция воспитателя зависит от истории полученных оценок, т.е. от того, что случилось раньше

Состояние – класс эквивалентных предысторий



Множество историй бесконечное (счетное)
Среди них есть и эквивалентные (с точки зрения одинакового дальнейшего поведения системы)

Реакция системы на очередной входной сигнал $b \in \Sigma$, если ее предыстория была aa, состоит из некоторого выхода И изменения истории: новая история станет aab

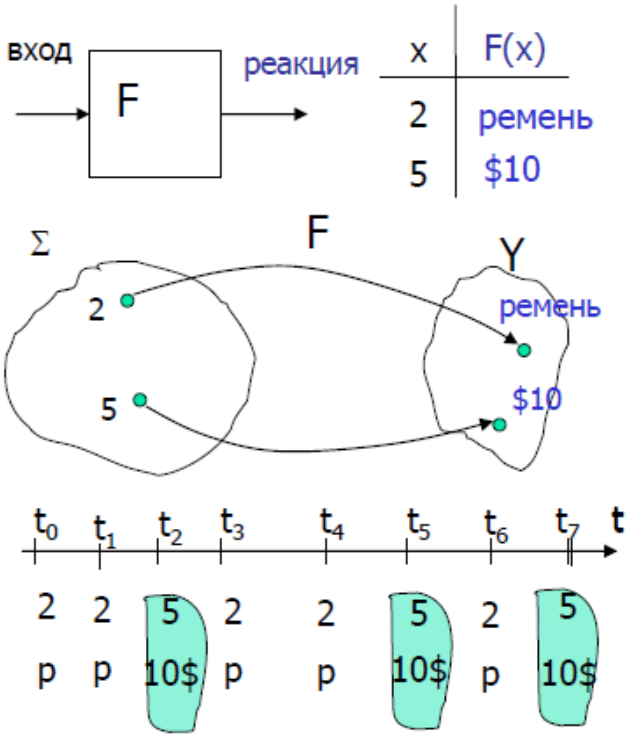
Назовем класс эквивалентных предысторий состоянием. Класс предысторий, в котором находится ПУСТАЯ ИСТОРИЯ ε, является начальным состоянием

Пусть S- множество классов эквивалентных предысторий системы (множество состояний). Тогда на каждый входной сигнал система с памятью будет выдавать выход (функция выхода λ: $S \times \Sigma \to Y$) и изменит свое состояние (функция перехода δ: $S \times \Sigma \to S$)

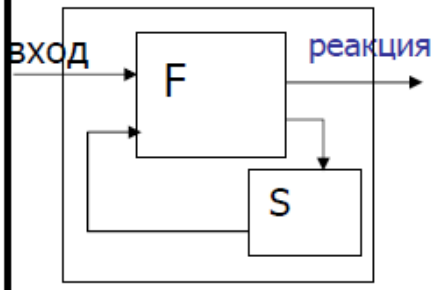
Обычно системы, с которыми мы имеем дело, конечные. Т.е. число классов эквивалентных предысторий таких систем конечно

Модель поведения при “Умном воспитании” школьника

Грубое воспитание - функция



Реакция на один и тот же вход зависит от истории и может быть разной



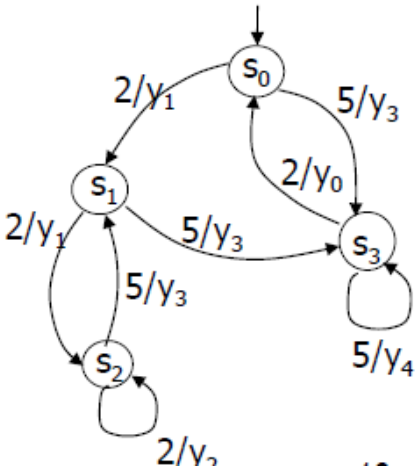
Что помним (состояния):

s_0 – *надежда (а вдруг?)*
s_1 – *беспокойство (неужели?)*
s_2 – *разочарование (дурак!)*
s_3 – *радость (сын не дурак!)*

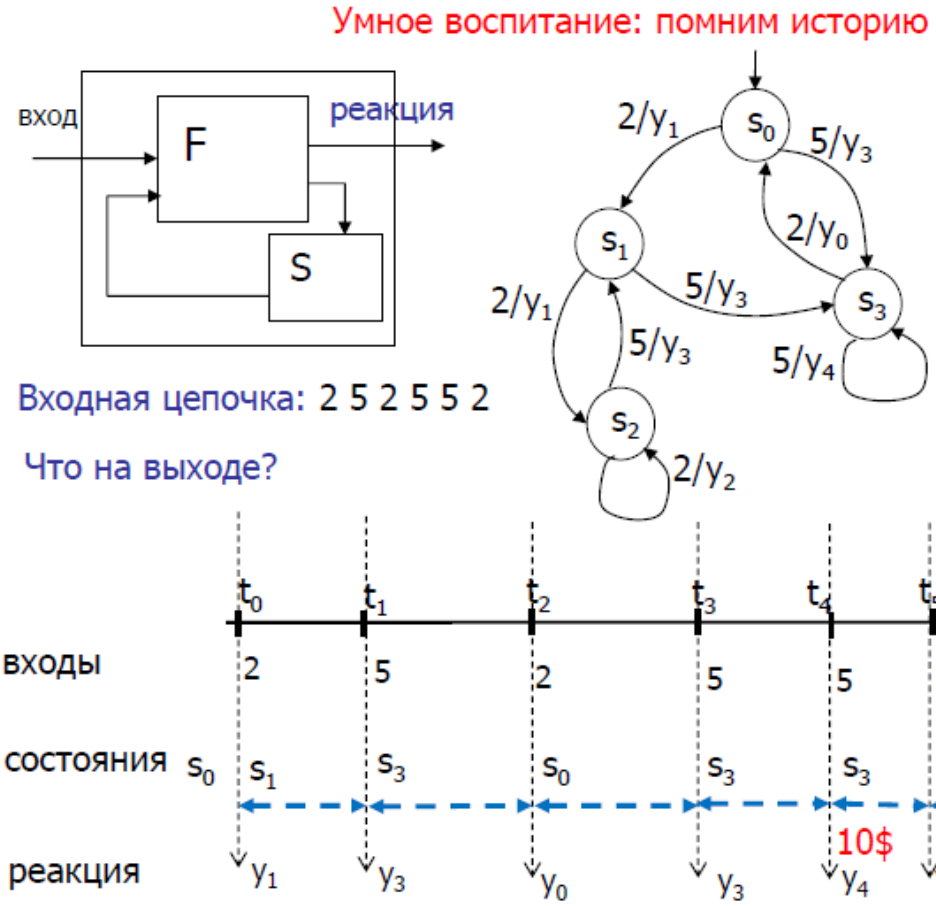
Возможные реакции:

y_0 – *успокаивать*
y_1 – *стыдить*
y_2 – *пороть*
y_3 – *хвалить*
y_4 – *дарить 10$*

$\delta: S\times\Sigma\to S$; $\lambda: S\times\Sigma\to Y$



“Умное воспитание” школьника – одна из траекторий



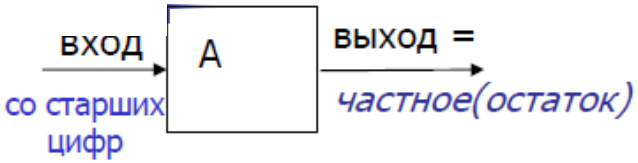
Что мы помним:

s_0 – *надежда (а вдруг?)*
s_1 – *беспокойство (неужели?)*
s_2 – *разочарование (дурак!)*
s_3 – *радость (сын не дурак!)*

Возможные реакции:

y_0 – *успокаивать*
y_1 – *стыдить*
y_2 – *пороть*
y_3 – *хвалить*
y_4 – *дарить 10$*

Пример КА: деление на 3



835 : 3 = 278 (1)

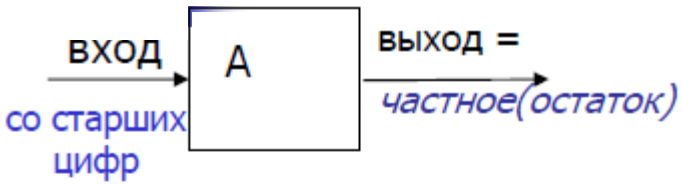
Предыстории преобразователя – цепочки десятичных цифр: 45, 6487, 032, ε (пустая история).
Какие предыстории (цепочки цифр) для этого устройства являются эквивалентными?
Те, после которых нужно запоминать одно и то же.

- Невозможно построить функциональный преобразователь
- Основная идея:
  в преобразователе столько состояний, сколько разных классов предысторий (эквивалентных входных цепочек) нужно запомнить

Что нужно запоминать в процессе деления 835 на 3?
Делим 8 на 3 => 2 и 2 в остатке. Нужно запомнить 2
Очередная входная цифра 3.
Делим 23 на 3 => 7 и 2 в остатке. Нужно запомнить 2.
Очередная входная цифра 5.
Делим 25 на 3 => 8 и 1 в остатке. Нужно запомнить 1.
Истории (цепочки цифр) эквивалентны, если у них один и тот же остаток от деления на 3.
Им соответствует одно и то же состояние

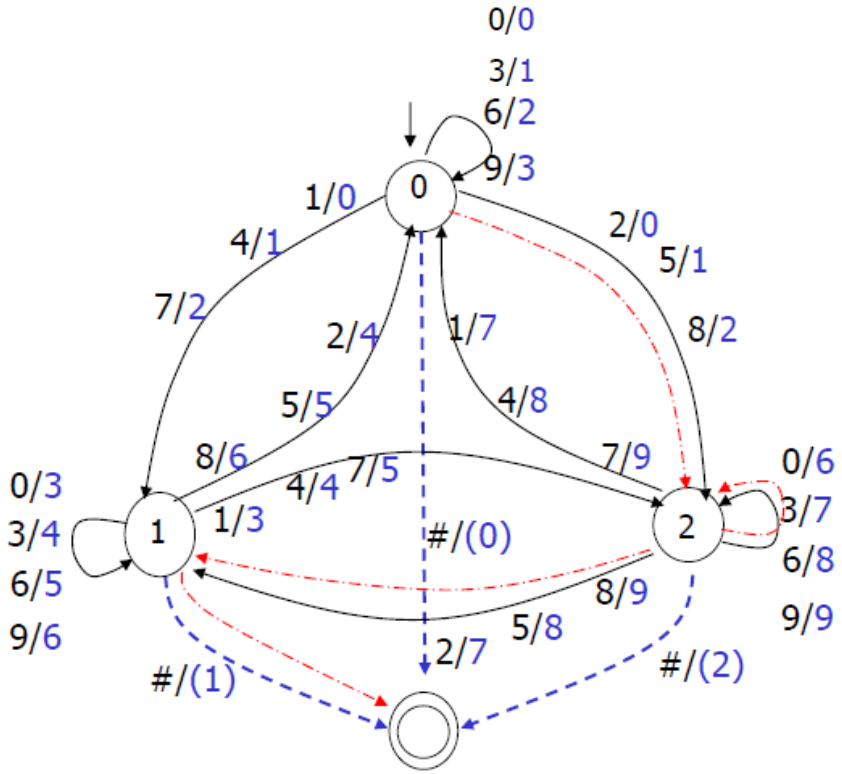
Состояния в этом автомате – классы эквивалентности входных цепочек: ОСТАТКИ от деления начальной части входной цепочки на 3. Таких остатков всего 3: это 0, 1 и 2

Пример КА: деление на 3



835 : 3 = 278 (1)

- Основная идея:
  в преобразователе входной информации столько состояний, сколько разных классов предысторий (классов эквивалентных входных цепочек) нужно запомнить *(имитация деления в столбик)*

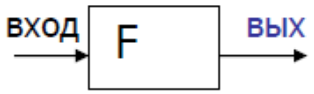


Цепочки 4, 7, 835 и 76 – эквивалентны: всем им соответствует состояние 1

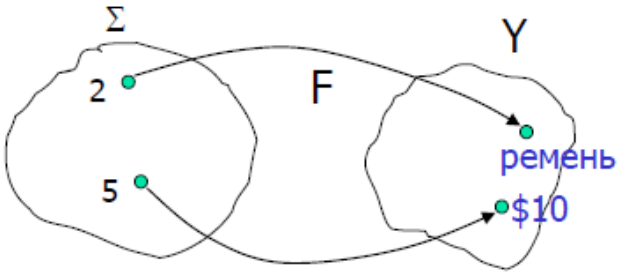
Вход 835# Выход 278 (1)

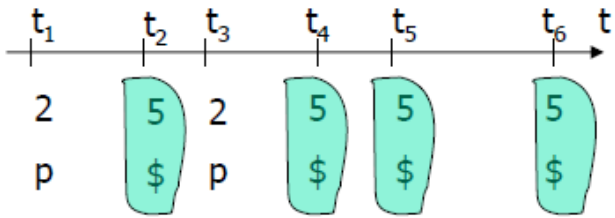
Пример: проблема воспитания

Грубое воспитание – функция:
принес 2 – ремень,
принес 5 - 10$



x	F(x)
2	ремень
5	$10

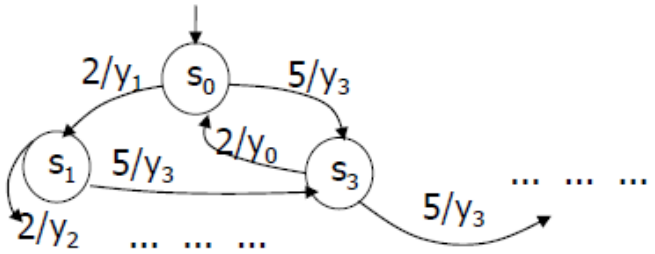




Умное воспитание – с памятью (помним историю!):

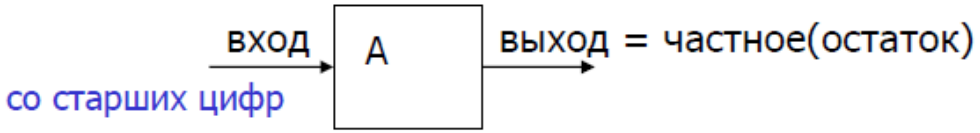
Принес 2 – не пороть сразу,
- если это случайная 2, то успокоить,
- если две подряд - начать беспокоиться,
- если несколько двоек, то ремень

Принес 5 – не награждать сразу,
- если это случайная 5, то просто порадоваться,
- если две подряд - хвалить,
- если уже несколько пятерок, то одаривать 10$



Умное воспитание состоит в том, что реакция на один и тот же вход разная в зависимости от истории

Пример автомата: деление на 3 в четверичной системе счисления

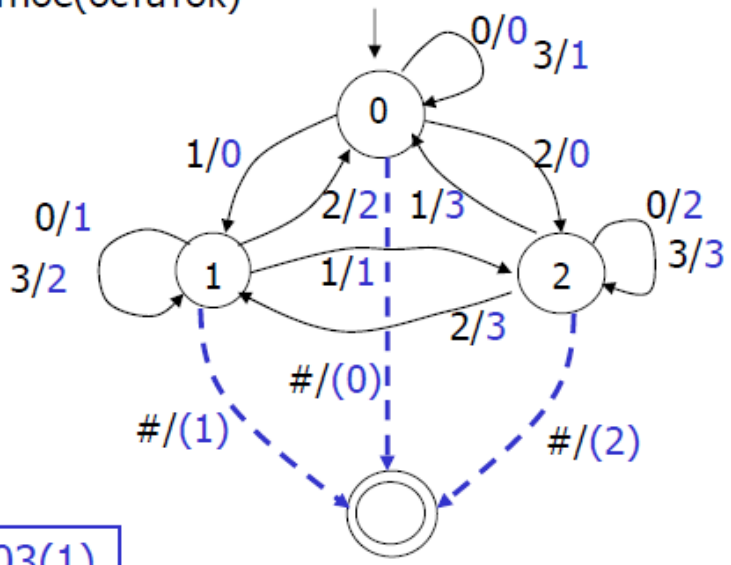


$21022_4 : 3 = ??$

$21022_4 = 586_{10}$ в десятичной системе

$586 : 3 = 195\ (1)$

$195_{10} = 03003_4$



Ответ на вход 21022# должен быть 3003(1)

- Основная идея: в преобразователе входной информации столько состояний, сколько разных классов предысторий мы должны запомнить.
- Состояния определяют, какие ОСТАТКИ мы запоминаем от деления любой начальной части входной цепочки на 3. Таких остатков всего 3: это 0, 1 и 2

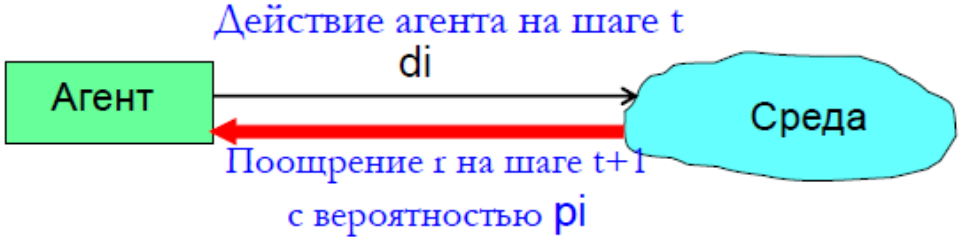
Автоматная модель целесообразного поведения

- ***Адаптация к среде***



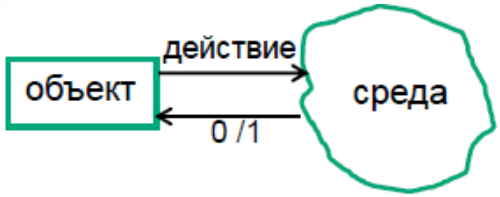
Целесообразное поведение в стационарной случайной среде

- Простейшая постановка задачи:
  - Агент выбирает свои действия из конечного множества D={d1, …, dn}
  - Оценка выбранного действия происходит немедленно: пусть это 0 или 1
  - Среда стохастическая: на действие di среда отвечает поощрением с вероятностью p_i (или, что то же, наказание с вероятностью 1-pi)
  - Среда стационарная: эти вероянтности не меняются со временем



- **ВОПРОС:** Как, не имея никакой первоначальной информации об особенностях среды, агент в процессе взаимодействия с окружающим миром может выработать целесообразное поведение в нем?
- Ясно, что модель – с памятью

Метафора: многорукий бандит



Действия: D={d1, …, dn}

Выигрыши (reward) с вероятностями R={p1, …, pn}

Выигрыш (reward) = 1, не выигрыш = 0

Действие di – это выбор бандита Bi

P1= 0.2 P2= 0.5 P3= 0.8



Бандит Bi дает reward с вер Pi

Не целесообразное поведение, если любое действие - с одинаковой вероятностью

Тогда средний выигрыш M = Σ(1/n * Pi) = 1/n * ΣPi

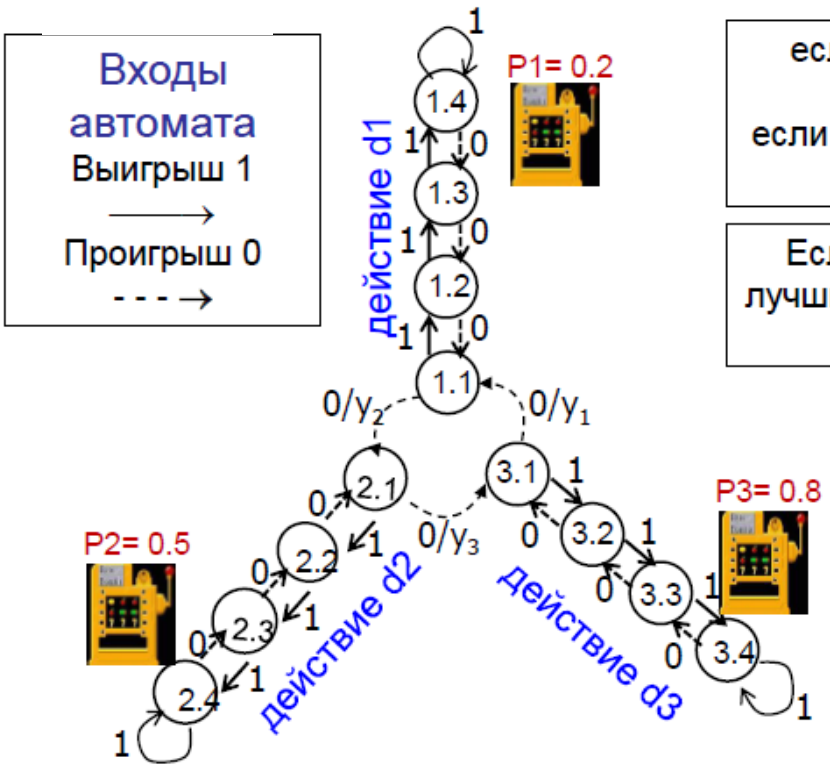
Если средний выигрыш больше M, то – поведение можно считать целесообразным

В нашем конкретном случае
Средний случайный выигрыш
M = 1/3(0.2+0.5+0.8) = 0.5

Если заранее знаем (“святым духом”), какой бандит лучший, то выигрыш = 0.8

Какая простейшая модель может обеспечить целесообразное поведение в широком диапазоне таких стационарных сред?

Автоматы с линейной тактикой



если выигрыш, то движемся вглубь по →, оставаясь на ветке,
если проигрыш – наружу по – –>(переходим, в конце концов, к другому бандиту)

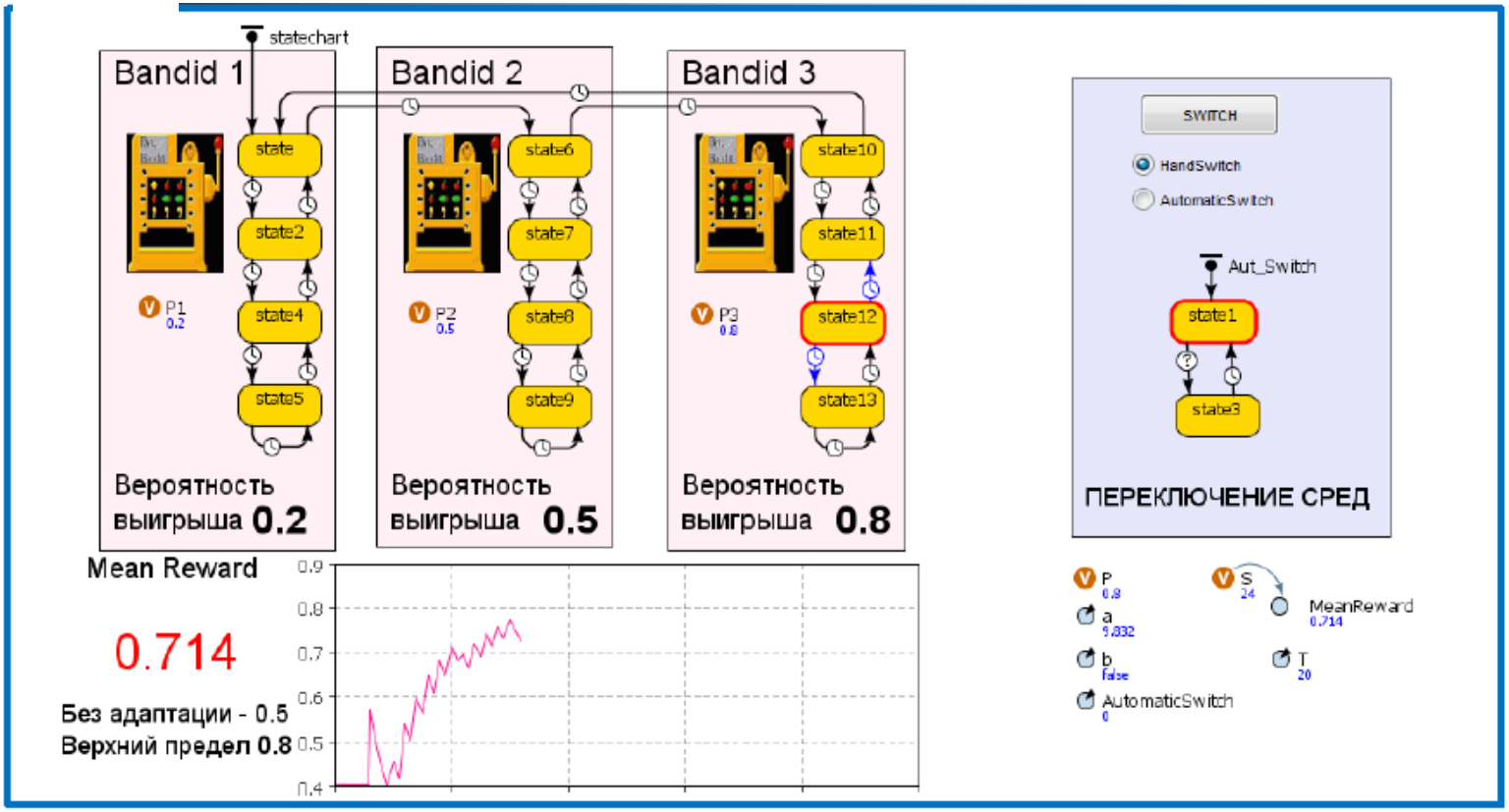
Если узнаем *святым духом*, какой бандит лучший, то выигрыш (за 1 шаг) = 0.8, работаем только с d3

Глубина памяти q влияет на "коэфф" целесообразности + +
инерционность - -

РЕЗУЛЬТАТЫ МОДЕЛИРОВАНИЯ
4 состояния: Mean Reward = 0.798
3 состояния: Mean Reward = 0.777
2 состояния: Mean Reward = 0.733
1 состояние: Mean Reward = 0.635

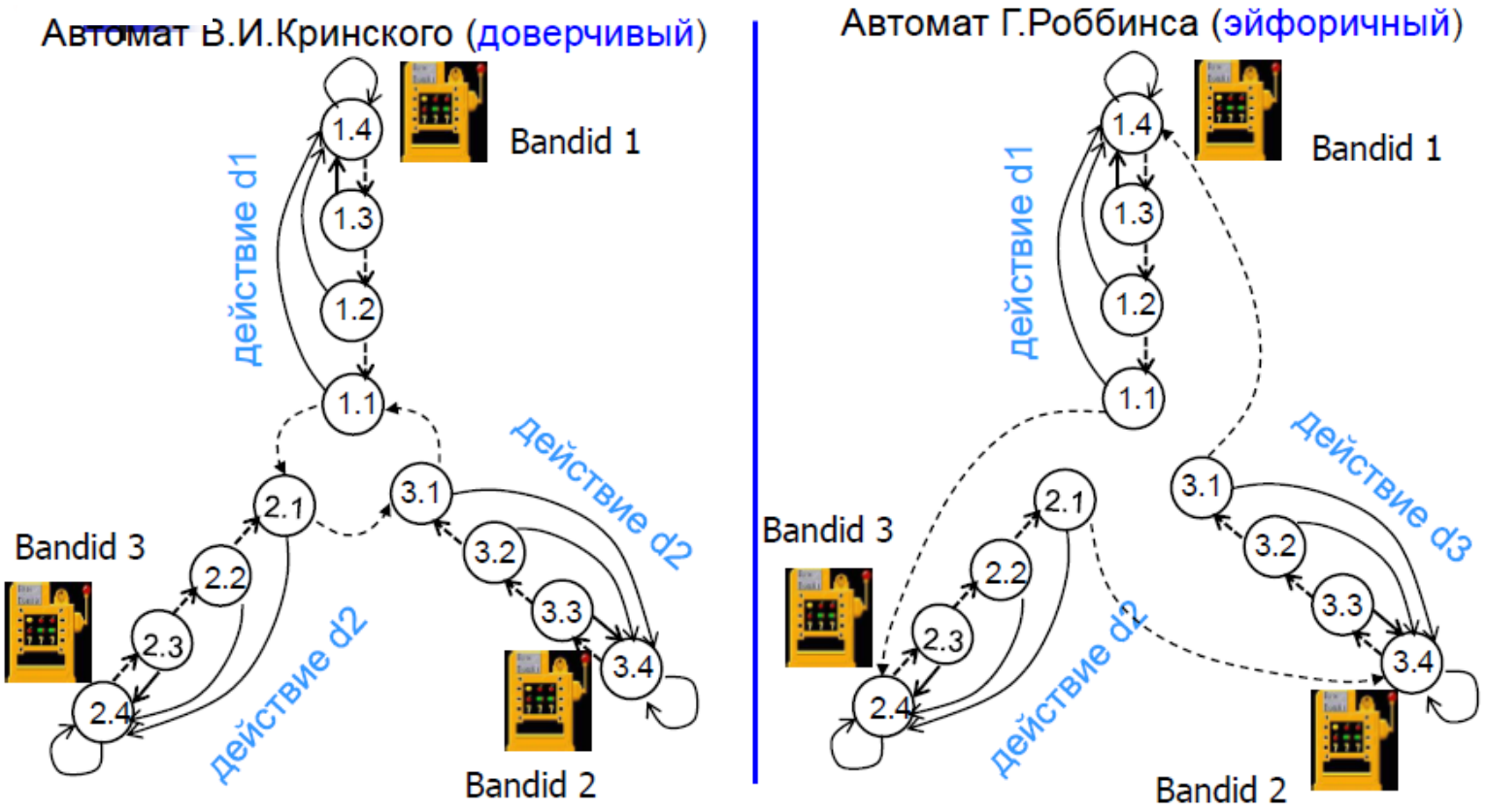
- Конструкция асимптотически оптимальна: при $q \to \infty$ выигрыш $\to$ Max(pi)

Работа имитационной модели автомата



- Работа автомата с линейной тактикой в игре с тремя однорукими бандитами. Если среда меняется, то большое число состояний вредит (консерватор в меняющейся ситуации проигрывает)

Другие асимптотически оптимальные автоматы



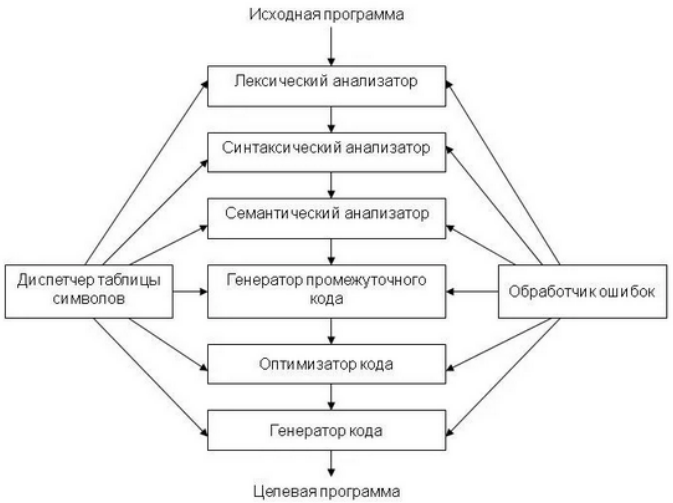
- Конструкции асимптотически оптимальны: при $q \to \infty$ выигрыш $\to$ Max(pi)

Абстрагирование в модели конечного автомата

- Считаем, что любое конкретное событие в жизни автомата происходит спонтанно. Механизм и причины возникновения событий на входе конечного автомата не рассматриваются, они вне теории
- Событие мгновенно, т. е. является элементарным действием, не имеющим протяженности во времени – ясно, это абстракция, ее нужно РЕАЛИЗОВАТЬ
- Если входное действие протяженное, т. е. требующее времени, его следует рассматривать как пару событий, первое из которых отмечает начало действия, а второе – его завершение
- Мы рассматриваем асинхронные системы, абстрагируемся от точной привязки входных событий ко времени. Преимуществом такого подхода является упрощение построений и рассуждений, их можно применить к физическим и вычислительным системам любой скорости, сложности и производительности
- Когда временная привязка событий необходима по существу, соответствующее рассмотрение проводится дополнительно



Конечные автоматы (детерминированные)



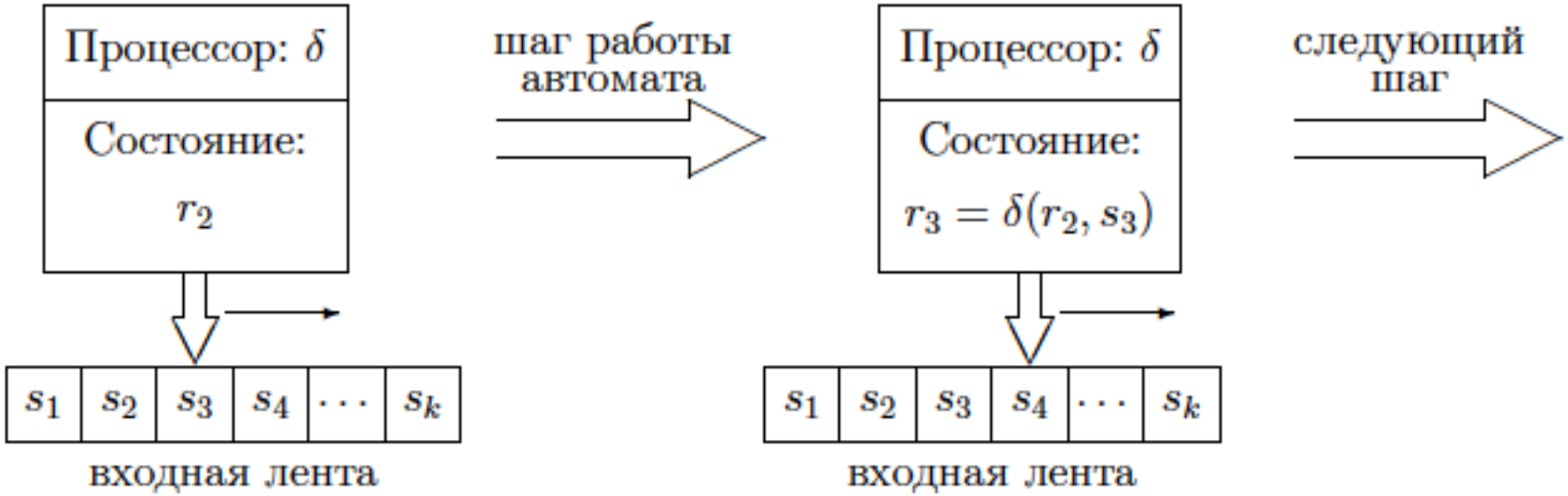
- Конечные автоматы относятся к классу *словарных алгоритмов*. С помощью конечных автоматов можно решать задачи из достаточно широкого семейства алгоритмических проблем.

Например,

- *технология проектирования микросхем основана на результатах теории автоматов.*
- *компиляторы, перерабатывающие текст программы, написанной на языке программирования высокого уровня, в программу на машинном языке.*
- Две стадии работы компилятора:
  - лексический анализ текста, который реализуется с помощью определенного конечного автомата.
  - синтаксический анализ, который осуществляется с помощью автомата с магазинной памятью.

- **Не любая** алгоритмическая задача поддается решению с помощью конечного автомата.

Неформальное определение КА



- ***Входными данными*** любого конечного автомата являются слова в некотором заранее фиксированном алфавите, ***выходными данными*** являются две логические константы *true* и *false*.
- В каждый момент времени автомат находится в определенном *внутреннем состоянии*. Число состояний автомата конечно.
- В начальный момент времени автомат находится в *начальном состоянии*.
- Некоторые состояния автомата считаются *выделенными*.

Принцип работы КА (неформально)

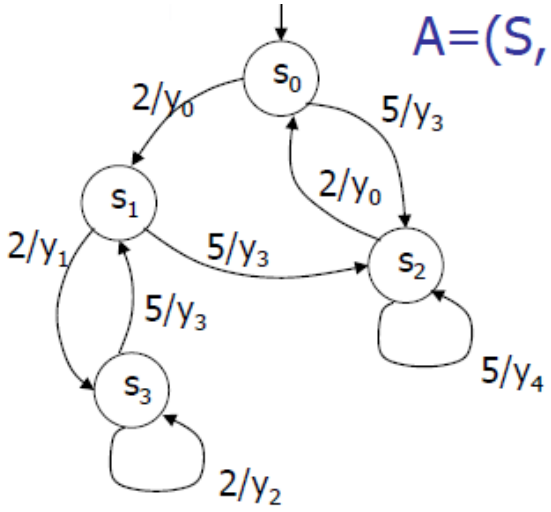
- Входное слово записывается на *входной ленте*, разбитой на ячейки: в каждой ячейке содержится одна буква слова.
- Автомат связан с лентой посредством специальной *управляющей головки*, которая последовательно считывает символы из ячеек, двигаясь слева направо. Очередной считанный символ отправляется в *процессор*, который по текущему состоянию и по данному считанному символу определяет *новое состояние*, в которое переводится автомат.
- Функция, вычисляющая это новое состояние называется *функцией перехода*. Функция перехода расположена внутри процессора и не изменяется в ходе вычислений.
- Считав все символы слова, автомат *останавливается* в определенном состоянии: если это состояние оказывается выделенным, то на выход подается константа ***true*** — автомат распознал слово, в противном случае подается константа ***false*** — автомат не распознал слово.

Свойства КА

- Из неформального определения видно, что конечные автоматы обладают такими интуитивными свойствами, как *конечность* описания (результативность), *дискретность*, *направленность* (элементарность шагов) и *массовость*.
- *Детерминированность* также присутствует в данном описании: по текущему состоянию и считанному символу функция перехода однозначно определяет новое состояние.
- Отличительной чертой конечных автоматов является ***отсутствие памяти вне процессора***, т. е. вся память автомата занята программой. Эта особенность позволяет пролить свет на причины *неспособности* конечных автоматов решить *любую* алгоритмически разрешимую задачу (для алгоритмов определенного вида необходима дополнительная память).



Конечный автомат – формальное определение



$A=(S, \Sigma, Y, s_0, \delta, \lambda)$

Для нашего примера

$S = \{s_0, s_1, s_2, s_3\}$
$\Sigma = \{2, 5\}$
$Y = \{y_0, y_1, y_2, y_3, y_4\}$
s_0 – начальное состояние ($s_0 \in S$)
$\delta: S \times \Sigma \to S$ $\quad \delta(s_0,2)=s_1,\ \delta(s_1,5)=s_2,\ \ldots$
$\lambda: S \times \Sigma \to Y$ $\quad \lambda(s_0,2)=y_0,\ \lambda(s_1,5)=y_3,\ \ldots$

S – конечное множество (состояний)

Σ – конечное множество (вх сигналов)

Y – конечное множество (вых сигналов)

s_0 – начальное состояние ($s_0 \in S$)

$\delta: S \times \Sigma \to S$ - функция переходов

$\lambda: S \times \Sigma \to Y$ – функция выходов

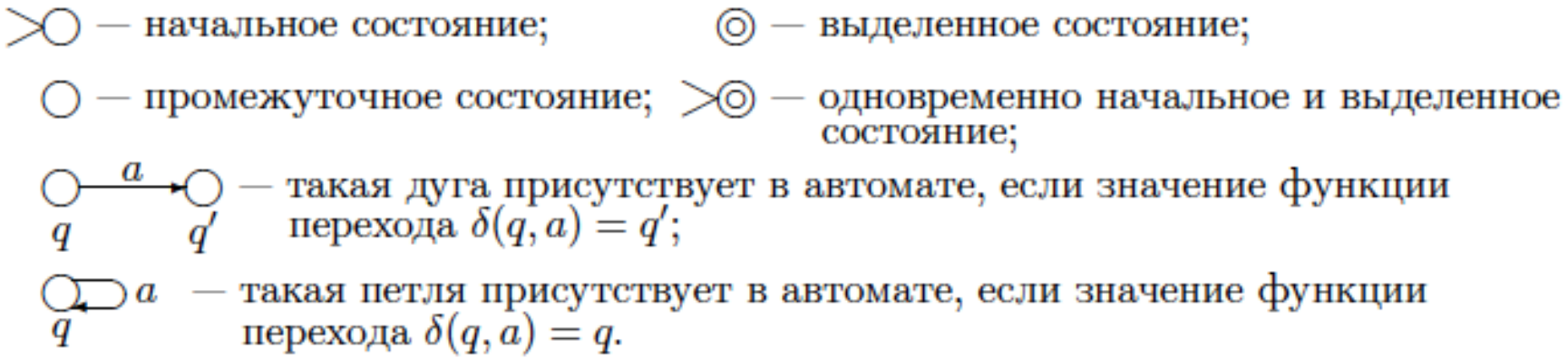
Таблицы переходов и выходов

состояния \ входы	δ	x=2	x=5
	s_0	s_1	s_3
	s_1	s_3	s_2
	s_2	s_0	s_2
	s_3	s_3	s_1

состояния \ входы	λ	x=2	x=5
	s_0	y_0	y_3
	s_1	y_1	y_3
	s_2	y_0	y_4
	s_3	y_2	y_3

Графическое изображение детерминированных автоматов

- Автоматы удобно изображать графически, используя следующие геометрические фигуры:



Пример

Автомат $A = \langle Q, A, \delta, q_0, F \rangle$, где

$A = \{0,1\}$,

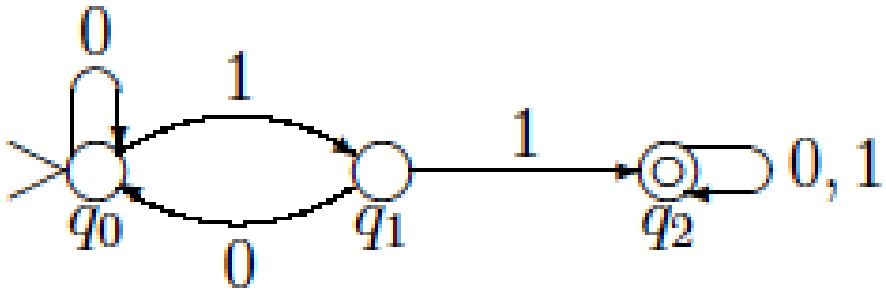
$Q = \{q_0, q_1, q_2\}$,

$F = \{q_2\}$,

а функция перехода задается соотношениями

$\delta(q_0, 0) = q_0$, $\delta(q_1, 0) = q_0$, $\delta(q_2, 0) = q_2$,

$\delta(q_0, 1) = q_1$, $\delta(q_1, 1) = q_2$, $\delta(q_2, 1) = q_2$,



КА и алгоритмы

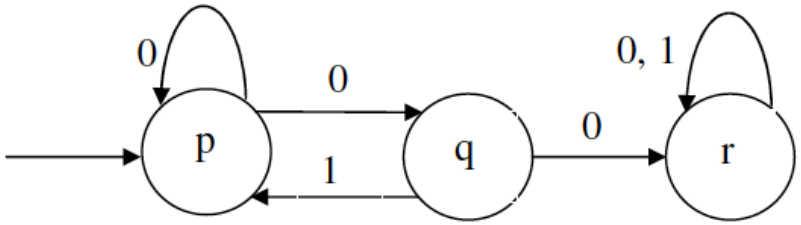
Конфигурация КА

- k=(q, ω), где q-текущее состояние КА, ω - непрочитанная цепочка символов слова на ленте, включая символ под читающей головкой.
- k = (q, ω) - текущая конфигурация;
  $k_0 = (q_0, \omega_0)$ - начальная конфигурация;
  $k_f = (q, e)$, где $q \in F$, - заключительная конфигурация и (e) – символ, обозначающий конец строки.

Шаг алгоритма - переход из одной конфигурации КА в другую

- Ki →Kj или (qi, ωi) → (qj, ωj).
- Функция переходов, заданная в табличной форме, может быть представлена графом переходов G=(Q, R), где Q - вершины графа, R - бинарное отношение между парой вершин, которое представлено множеством дуг (q_i, q_j).
- $(q_i, q_j) \in Q*Q$, если существует символ **a** $\in \Sigma$ и $\delta (q_i, a) = q_j$.
- На дугах графа (q_i, q_j) отмечаются соответствующие символы алфавита.

КА и алгоритмы



Q	Σ	
	0	1
p	q	p
q	r	p
r	r	r

Исполнение алгоритма –
это последовательность шагов,
в которых изменяется конфигурация КА

- (p,01001) →(q,1001) →(p,001) →(q,01) →(r,1) →(r,e);
  (p, 01001)- начальная конфигурация;
  (r, e) – конечная конфигурация.

- В результате применения слова ***01001*** в начальном состоянии **p** автомат переходит в следующее состояние **q** и следующее значение цепочки символов на входе ***1001***.

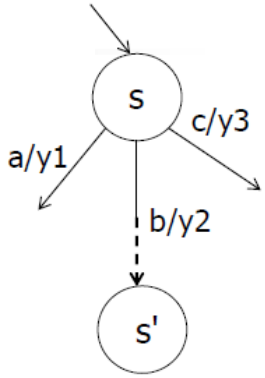
Автомат M допускает слово ω_0 , если существует

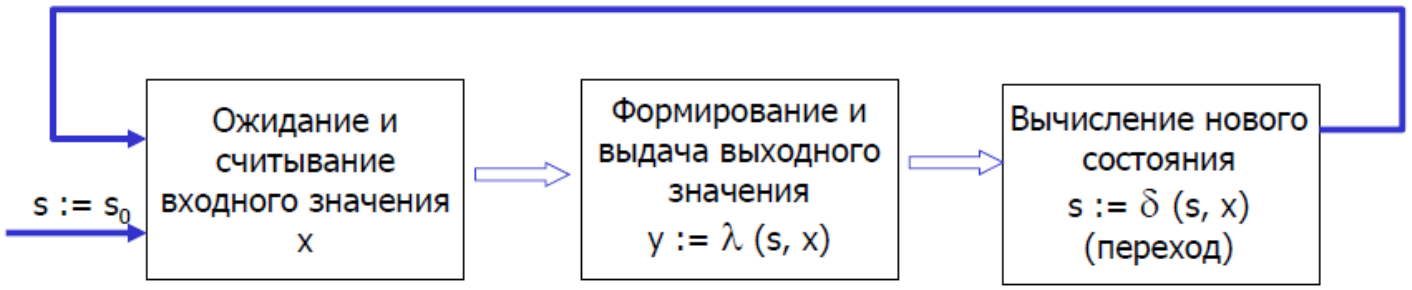
- $(q_0, \omega_0) \rightarrow^* (q_f, e)$,
- где $\rightarrow^*( )$ обозначает *транзитивное замыкание* и существует путь, соединяющий q_0 и q_f для входного слова ω_0.

Язык L(M), ***определяемый*** (распознаваемый, допускаемый) автоматом M включает множество всех слов, допускаемых M.

Программная реализация КА: язык ВУ

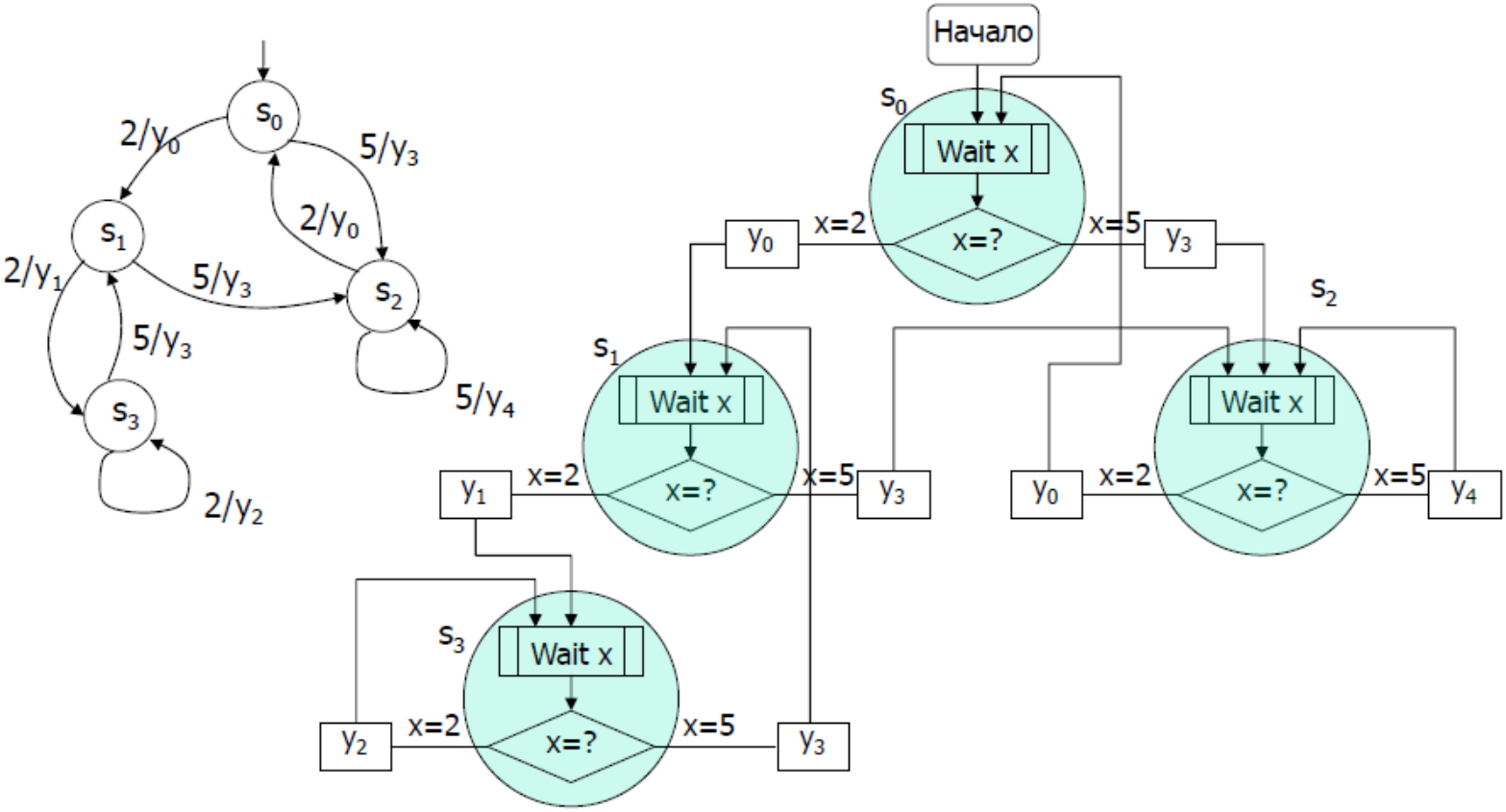
- Конечный автомат в каждом новом состоянии выполняет следующую последовательность действий:
  - ожидание и считывание входного значения
  - вычисление функции выхода и нового состояния по полученному входу и текущему состоянию
  - формирование (выдача) выходного значения





Программная реализация КА: блок-схемы

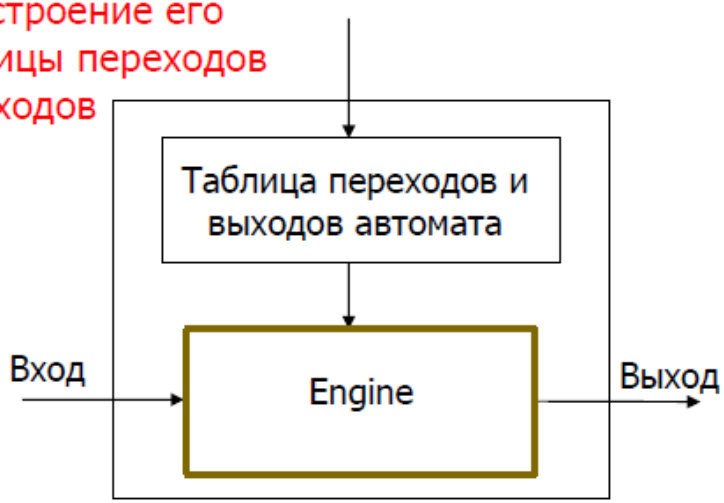
- Программная реализация КА может быть выполнена в любом языке программирования, в котором реализована операция "ожидание события", и в любой технологии



Программная реализация КА: программа с настраиваемой функцией переходов и выходов

- Реализаций КА может быть очень много, они почти очевидны
- Одна из часто используемых реализаций – такая:
  - строится engine, который хранит номер текущего состояния
  - таблица переходов и выходов настраивается для каждого конкретного автомата

Настройка автомата – построение его таблицы переходов и выходов



Engine работает с таблицей переходов и выходов, которая по номеру текущего состояния и входу возвращает пару: номер следующего состояния и номер действия (функция переходов и выходов)

Таблица переходов генерируется, перестраивается для каждого нового автомата,
Engine строится один раз

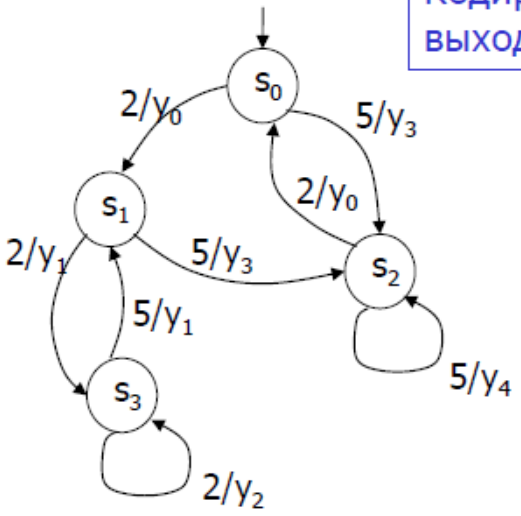
Аппаратная реализация конечного автомата

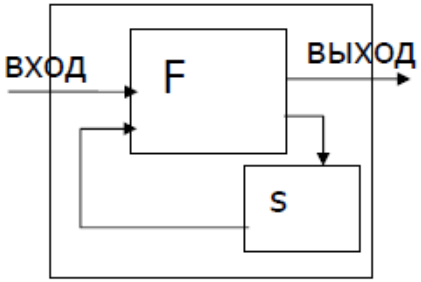
- Фактически, мы используем формальную модель для построения реальных алгоритмов и аппаратных устройств преобразования информации
- Аппаратная реализация КА проста и очевидна
- Она состоит из кодирования входов, состояний и выходов автомата и представления функций переходов и выходов в форме кодированной таблицы переходов и выходов
- Аппаратная реализация КА сводится к реализации двоичных функций и элементов памяти
- Важной проблемой при реализации КА является устранение гонок – подачи на вход автомата в текущем такте значения нового состояния, которое получается как выход функции переходов на текущем состоянии и текущем входном сигнале



Аппаратная реализация конечного автомата

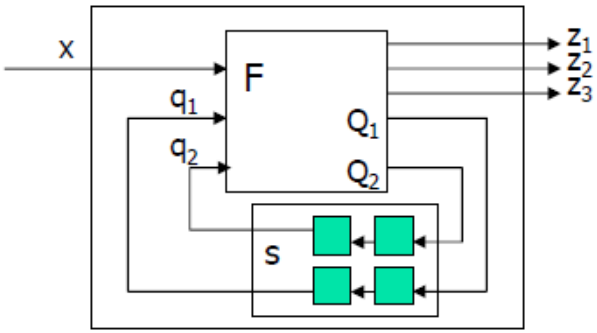
Кодируем входы, состояния и выходы двоичными кодами





Входных значений 2: значения двоичной переменной x
'2' кодируем x=0
'5' кодируем x=1

Состояний 4: q_1q_2
's_0' кодируем 00
's_1' кодируем 01
's_2' кодируем 10
's_3' кодируем 11

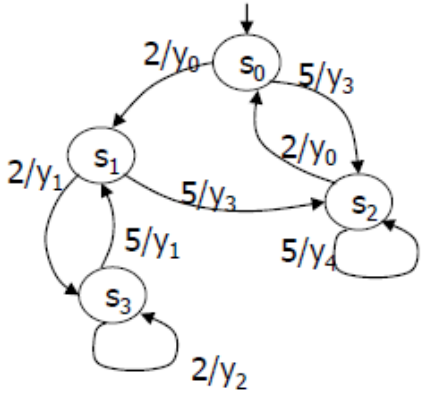


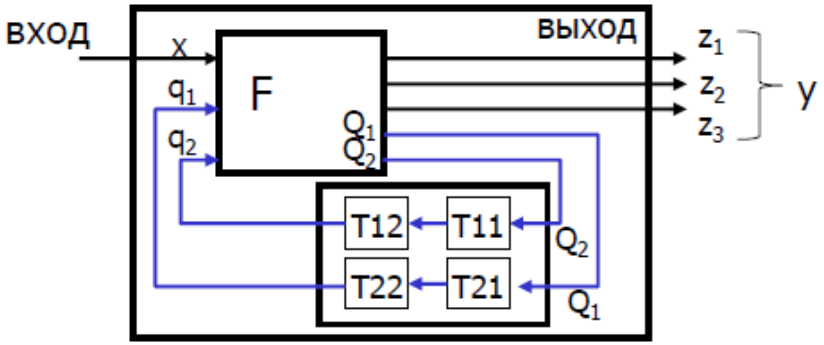
Выходов 5: $z_1z_2z_3$
'y_0' кодируем 000
'y_1' кодируем 001
'y_2' кодируем 010
'y_3' кодируем 011
'y_4' кодируем 100

Можем считать S, Σ, У переменными, а конкретные символы – значениями этих переменных

Аппаратная реализация конечного автомата

- Функциональный блок автомата реализует функции перехода и выхода



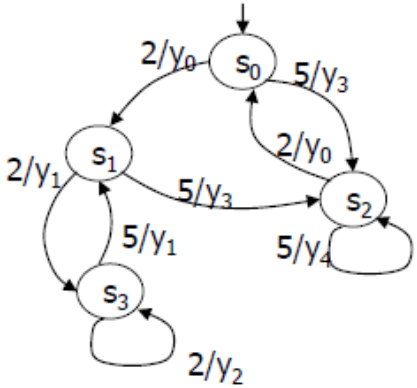


x s	s'	y
0 s0	s1	y0
0 s1	s3	y1
0 s2	s0	y0
0 s3	s3	y2
1 s0	s2	y3
1 s1	s2	y3
1 s2	s2	y4
1 s3	s1	y1

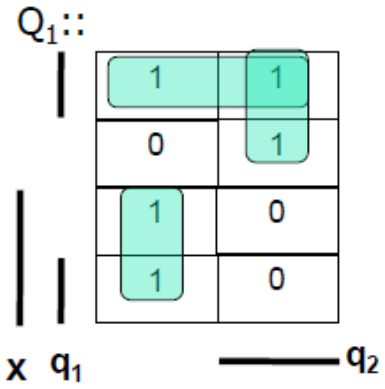
x q_1 q_2	Q_1	Q_2	z_1	z_2	z_3
0 0 0	0	1	0	0	0
0 0 1	1	1	0	0	1
0 1 0	0	0	0	0	0
0 1 1	1	1	0	1	0
1 0 0	1	0	0	1	1
1 0 1	1	0	0	1	1
1 1 0	1	0	1	0	0
1 1 1	0	1	0	0	1

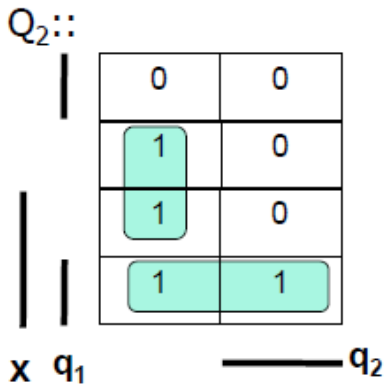
Аппаратная реализация конечного автомата

- Функциональный блок автомата реализует функции перехода и выхода



x q1 q2	Q1	Q2	z1	z2	z3
0 0 0	0	1	0	0	0
0 0 1	1	1	0	0	1
0 1 0	0	0	0	0	0
0 1 1	1	1	0	1	0
1 0 0	1	0	0	1	1
1 0 1	1	0	0	1	1
1 1 0	1	0	1	0	0
1 1 1	0	1	0	0	1





$Q_1 = x\neg q_1 \vee \neg\, xq_2 \vee x\neg q_2$

$Q_2 = \neg\, x\, \neg q_1 \vee q_1 q_2$

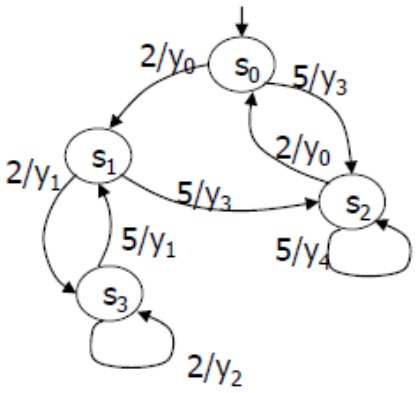
$z_1 = x\, q_1\, \neg q_2$

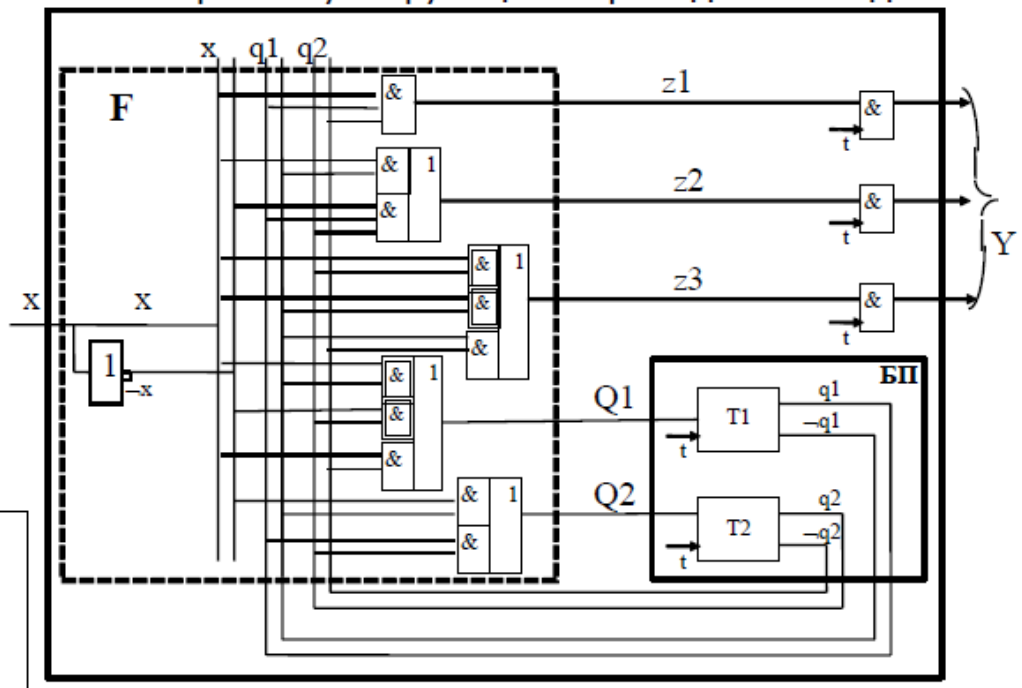
$z_2 = x \neg\, q_1 \vee \neg x\, q_1 q_2$

$z_3 = x\, q_2 \vee x\, \neg q_1 \vee \neg\, q_1\, q_2$

Аппаратная реализация конечного автомата

- Функциональный блок автомата реализует функции перехода и выхода





$Q_1 = x\neg q_1 \vee \neg x q_2 \vee x\neg q_2$

$Q_2 = \neg x\neg q_1 \vee q_1 q_2$

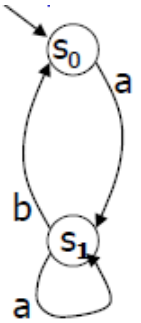
$z_1 = x\, q_1\, \neg q_2$

$z_2 = x\, \neg q_1 \vee \neg x\, q_1 q_2$

$z_3 = x\, q_2 \vee x\, \neg q_1 \vee \neg q_1\, q_2$

Реализация КА сводится к реализации логических функций и элементов памяти

Пример: символьное представление системы переходов (конечных автоматов)



Переходы

s_0 a s_1
s_1 a s_1
s_1 b s_0

Кодирование

s_0 – 0 s_1 – 1

a – 0 b – 1

z - Состояния кодируются одной двоичной переменной z.
z` - кодирует следующее состояние

x - Входные сигналы кодируются одной двоичной переменной x

Система задается всеми своими переходами

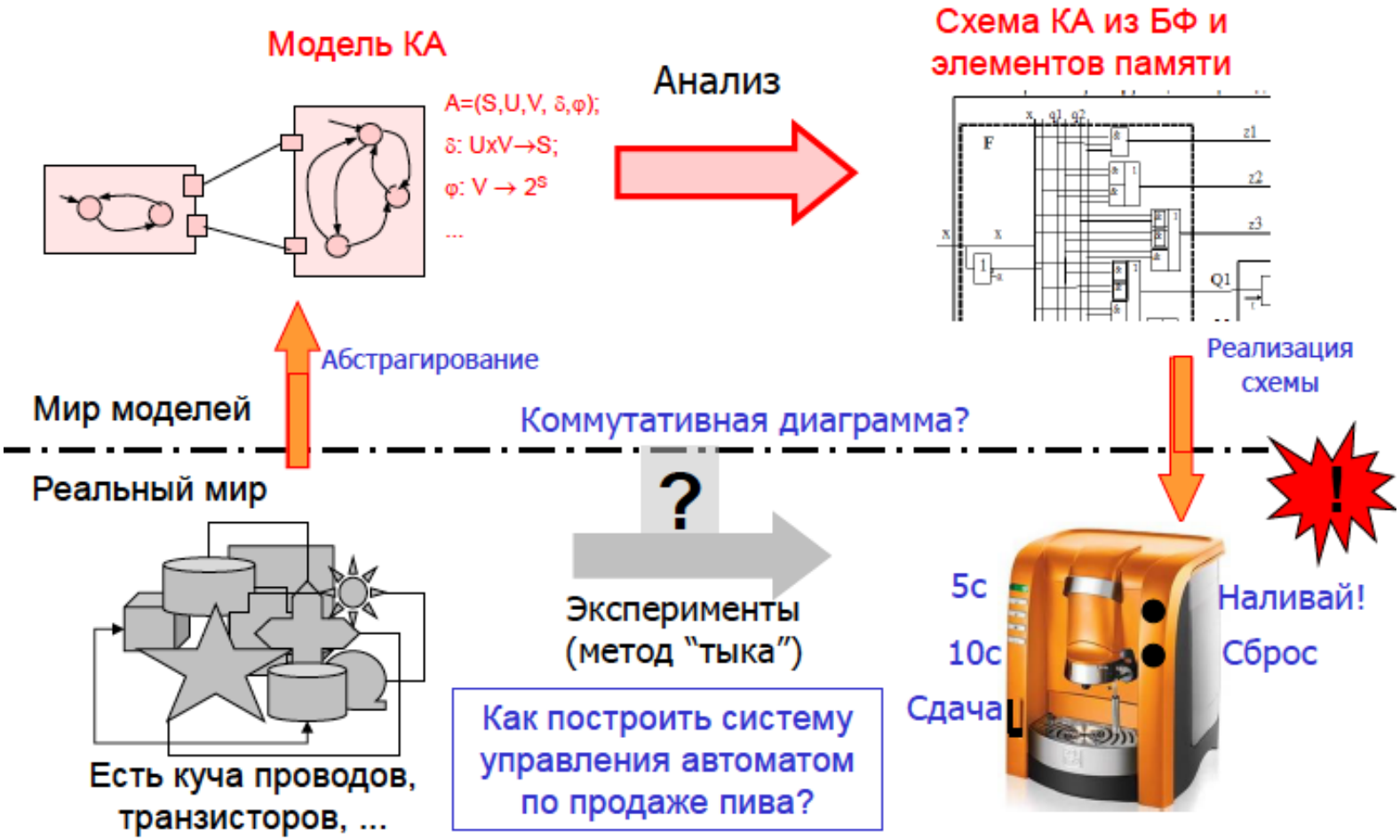
	z x z'	F
s_0 a s_0	0 0 0	0
s_0 a s_1	0 0 1	1
s_0 b s_0	0 1 0	0
s_0 b s_1	0 1 1	0
s_1 a s_0	1 0 0	0
s_1 a s_1	1 0 1	1
s_1 b s_0	1 1 0	1
s_1 b s_1	1 1 1	0

Вся система переходов задается двоичной функцией

$$F = \neg x z' \vee z x \neg z'$$

Двоичная функция F равна 1 на тех наборах переменных, которые кодируют переходы

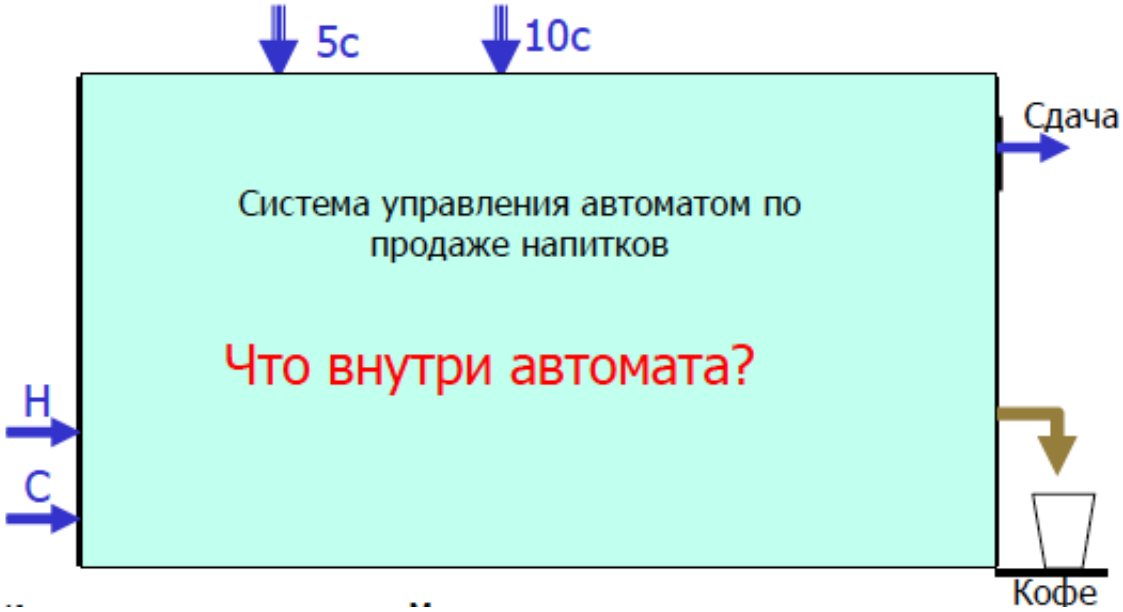
Использование модели КА в прикладных задачах



Пример: автомат по продаже напитков



Кофе стоит 15 центов, автомат принимает 5 и 10 центов



Пример: автомат по продаже напитков

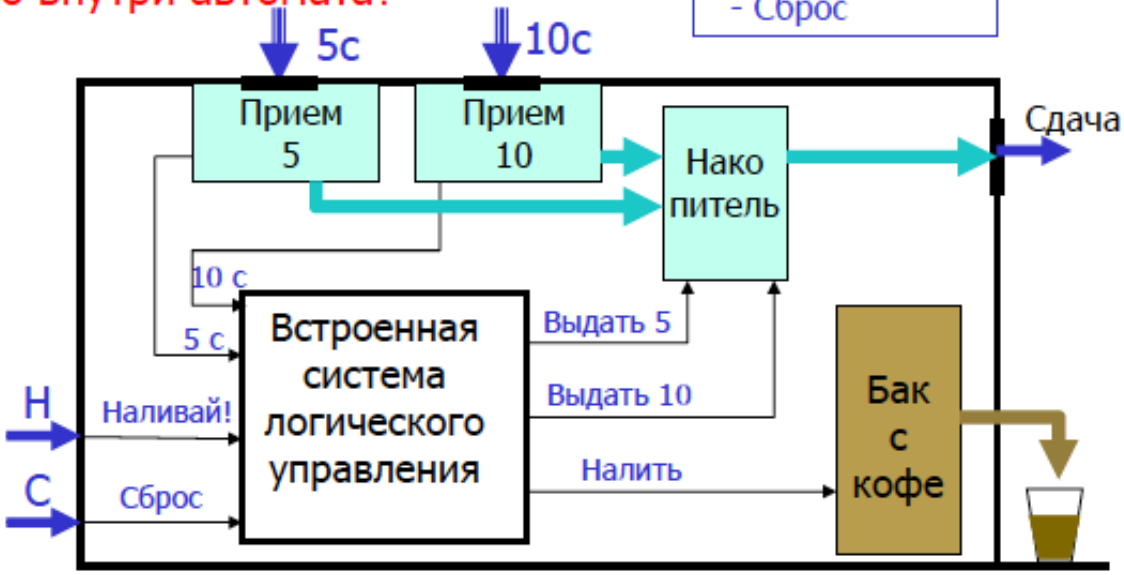


Кофе стоит 15 центов, автомат принимает 5 и 10 центов

Входы:
- принять 5 с
- принять 10 с
- Наливай!
- Сброс

Выходы:
- выдать сдачу 5 с
- выдать сдачу 10 с
- налить кофе

Что внутри автомата?



Конечный автомат – модель системы управления автомата по продаже напитков

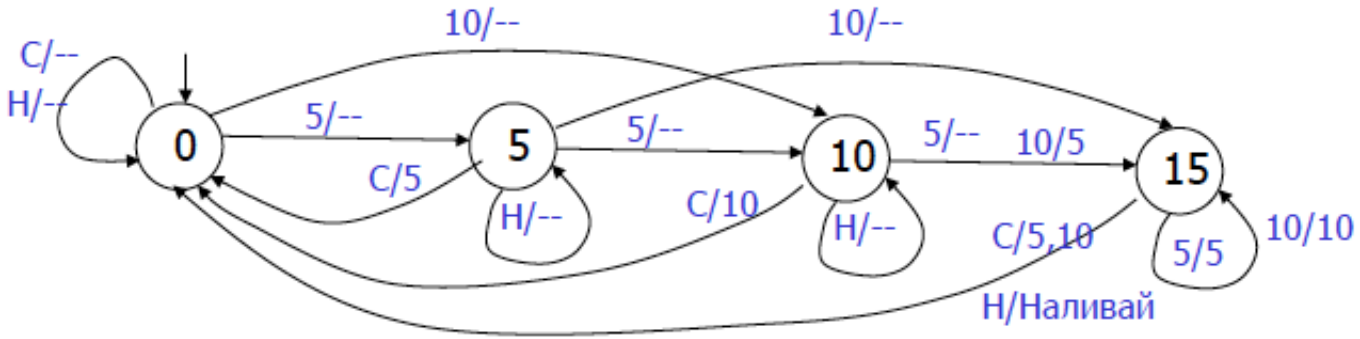


Реакции на "Наливай!" в цепочках Наливай!,Наливай; 5,5,10,Наливай! и 5,С,5,5,Н будут разными: СУ должна помнить историю

Как построить систему управления так, чтобы запомнить все возможные предыстории?

ИСПОЛЬЗОВАТЬ модель конечного автомата!!

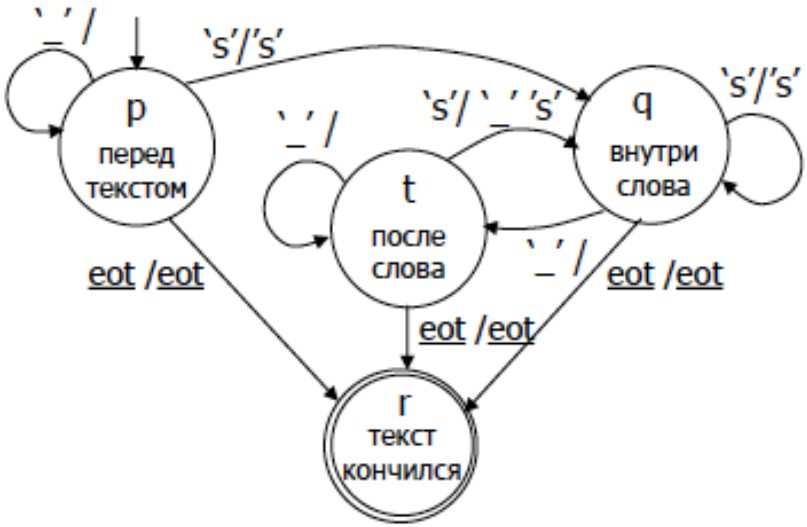
Каждое состояние системы управления запоминает класс эквивалентных предысторий. В качестве характеристики такого класса можно взять сумму, которую автомат задолжал покупателю



Построение программ, реализующих автоматное поведение (сначала – явное построение состояний)

- Программа выбрасывания пробелов из текста (s - значащий символ)

Пример
Вход: "_ _ Завтра_ _ _праздник _ _"
Выход: "Завтра_праздник"



Программу строим по автомату

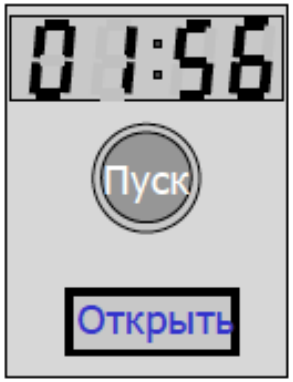
```
main() {
    char x, state = 'p' ; /* уст нач состояние*/
    while ( True )   {  wait (input);
      x = fgetc (input);
      switch (state)   {
         case 'p':
             switch(x)   {
                case 's' : output ('s'); state = 'q'; break;
                case '_'   : break;
                case 'eot': output ('eot'); state ='r'; break;
             } ; /* end switch x */
          break;
         case 'q':
            switch(x) {
              case 's': output ('s');  state = 'q'; break;
              case '_': output ('_');  state = 't'; break;
              case 'eot': output ('eot'); state ='r'; break;
            }; /* end switch x */
         break;
         case 't':
          ... ...
         break;
         case 'r':
          ... ... ...
         break;
       }; /* end switch state */
    };  /* end while */
}  /* end main */
```

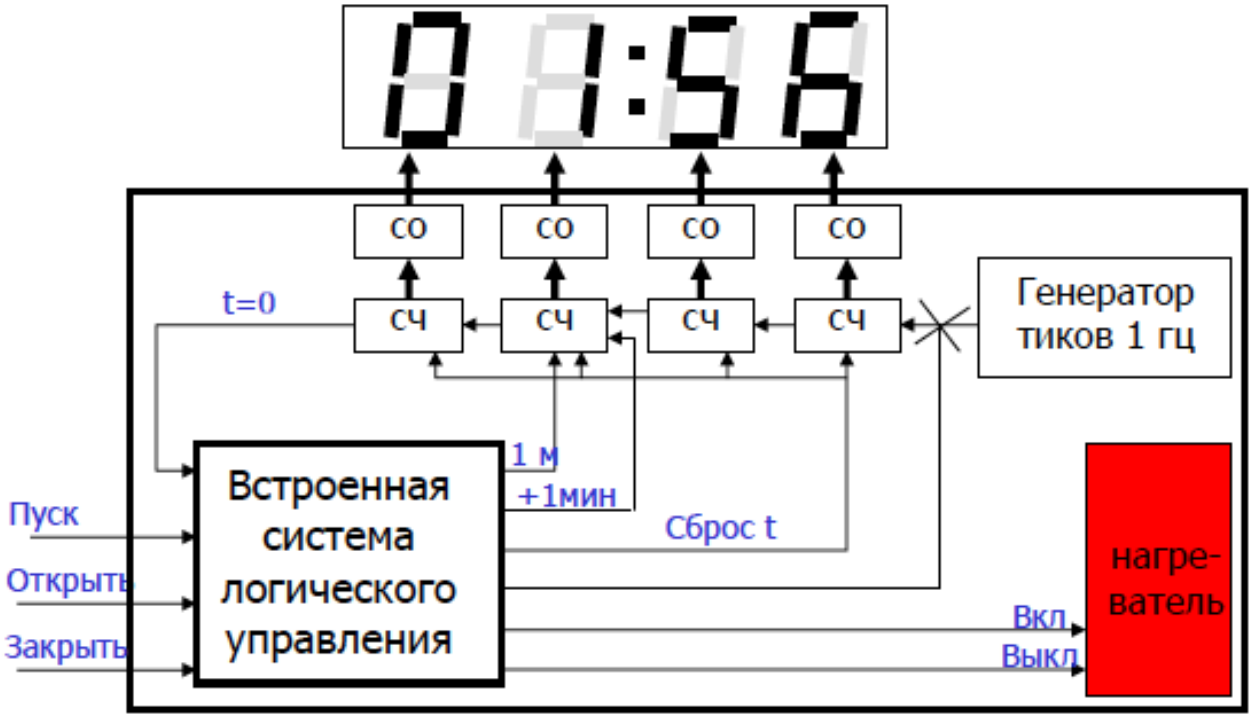
Система управления микроволновой печью

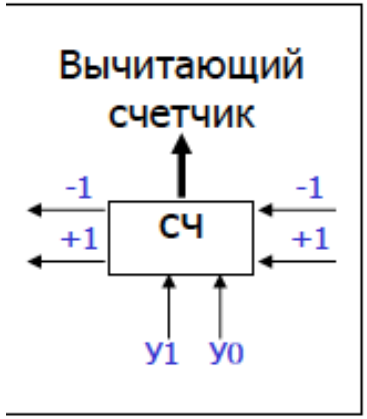


СУ микроволновкой
Redmond RM-1004

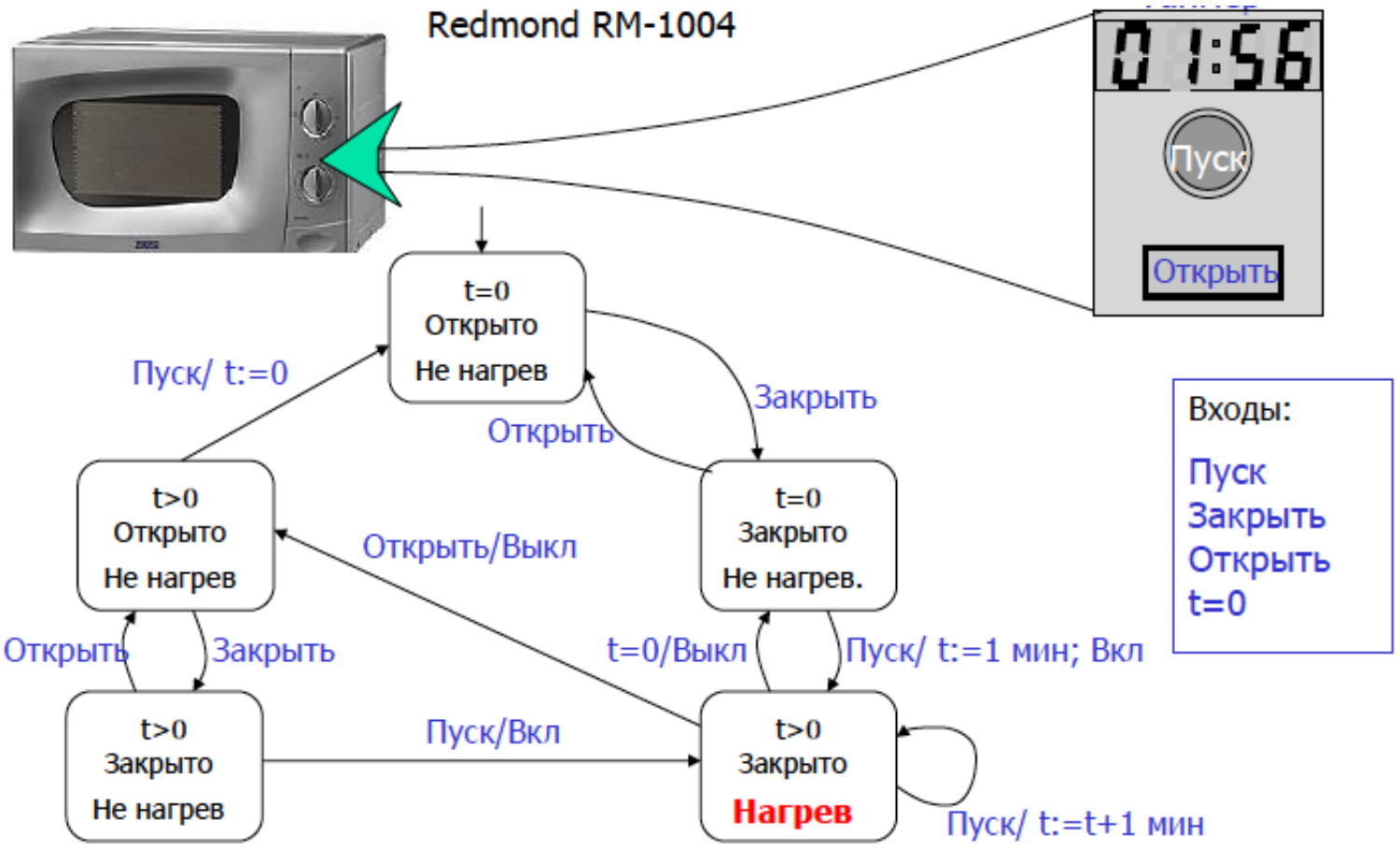
Таймер







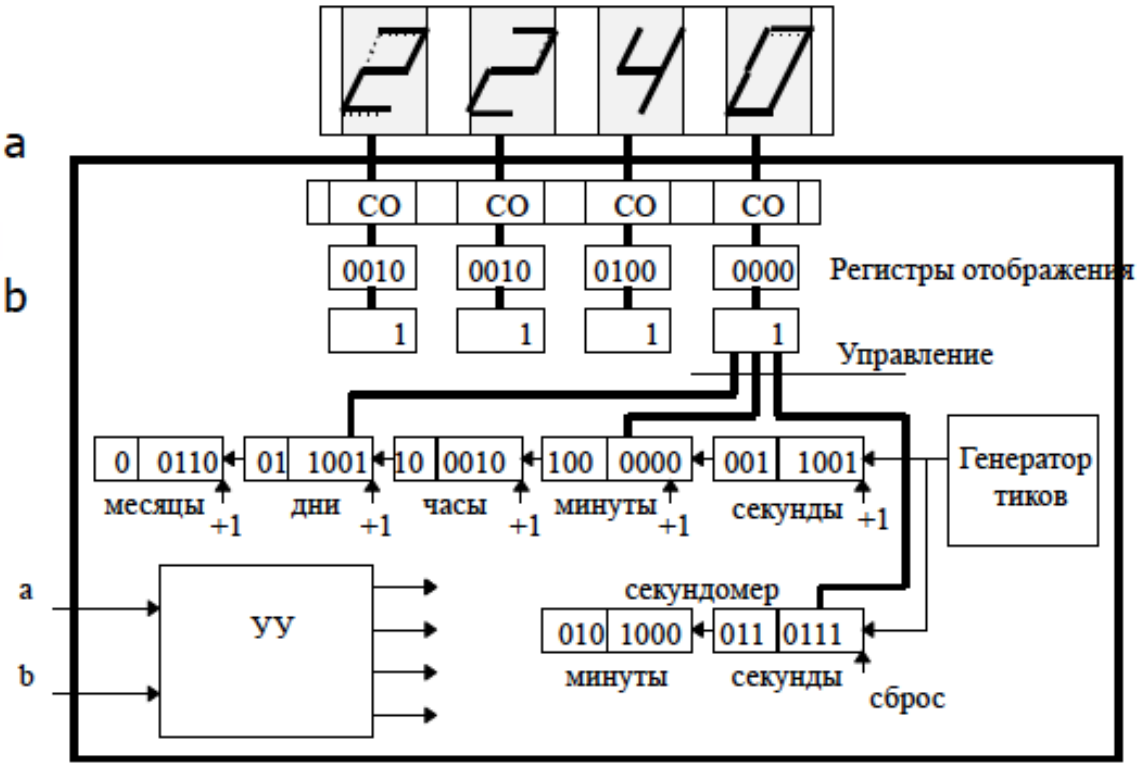
Система управления микроволновой печью



Структура электронных часов

- Система управления электронных часов позволяет выполнять множество функций

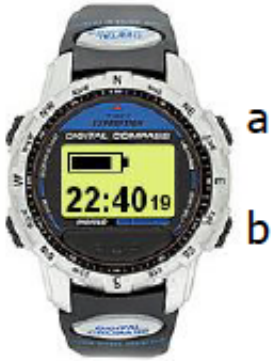




Система управления “умными” электронными часами

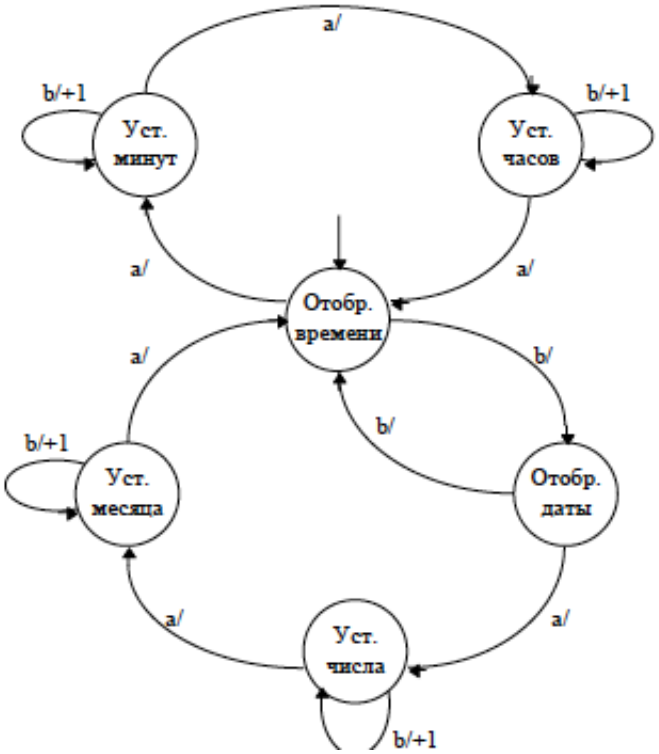
- “Smart” часы могут выполнять множество функций:

  - отображать время;
  - устанавливать время и дату;
  - отображать дату;
  - работать как секундомер;
  - ...



a

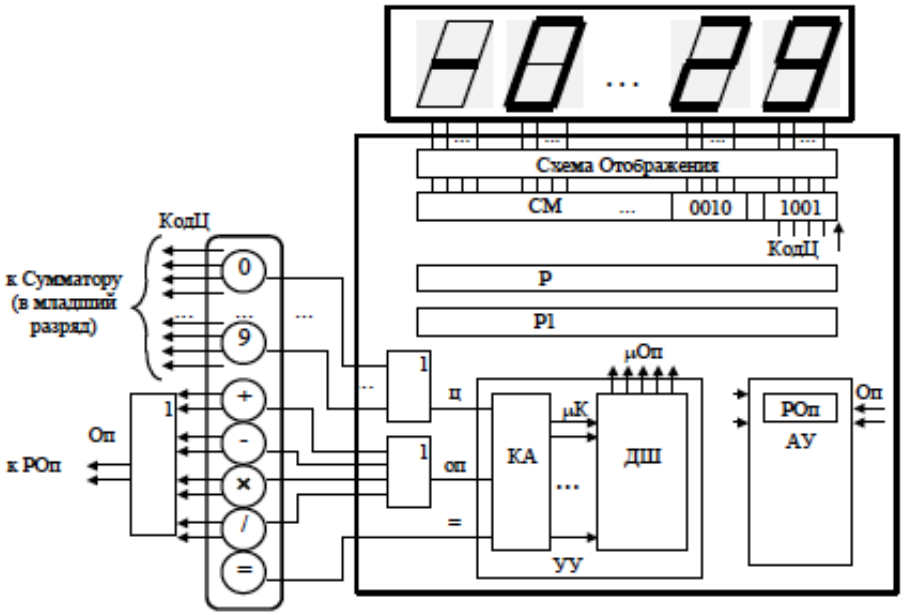
b

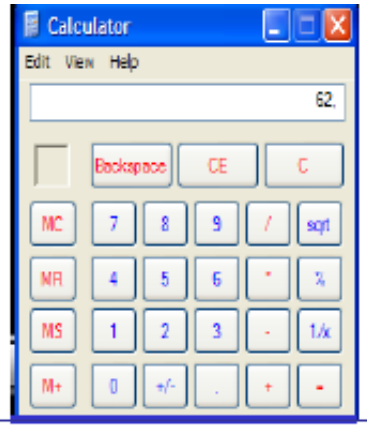


Система управления микрокалькулятором

'2+3*4 =' дает 20 слева направо без запоминания

Но что будет выдано на '2+3==*=' ? 64!!





СМ – Сумматор (регистр результатов)

Р, Р1 – Регистры

Роп – Регистр операций

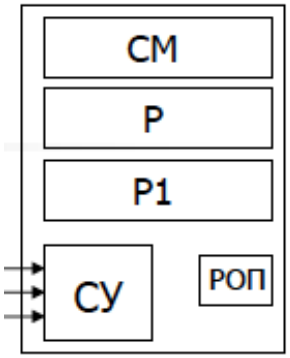
АУ – Арифметическое устройство

ДШ – Дешифратор

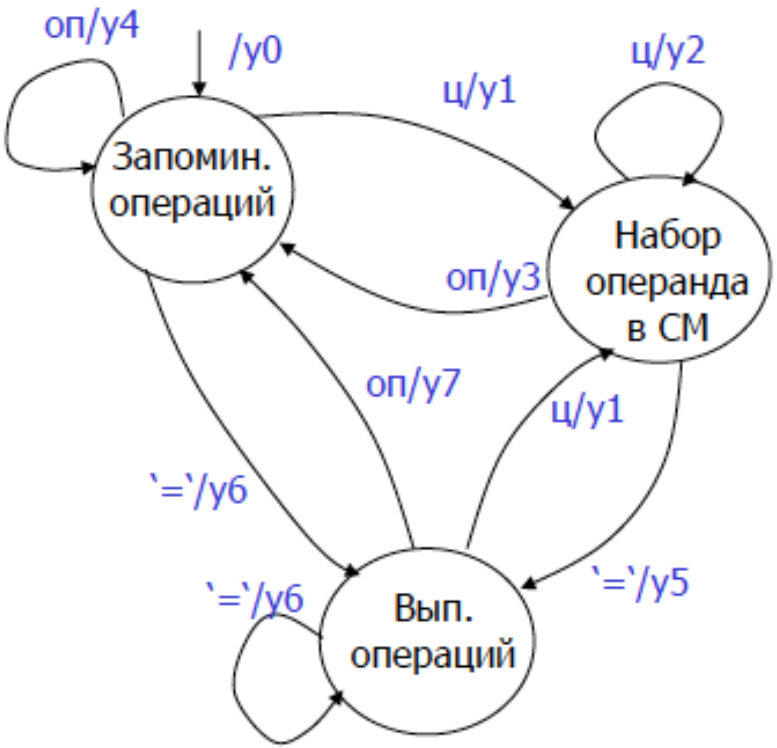
УУ – Устройство управления

- У каждого типа калькуляторов нестандартные последовательности символов обрабатываются по-своему

Конечный автомат управления микрокалькулятором



Встроенный калькулятор OS Windows XP



Входы: {оп, '=', ц}
Выходы: {y0, y1, y2, … , y7}

y0: 0 →СМ; 0→Р; 0 →РОП

y1: 0 →СМ; ц→$СМ_{1-4}$

y2: $СМ_{\leftarrow 4}$; ц→$СМ_{1-4}$

y3: <РОП>(Р, СМ)→СМ; СМ →Р; оп→РОП
(Первый операнд – в Р)

y4: оп →РОП

y5: СМ → Р1; <РОП>(Р, СМ) → СМ; Р1 → Р
(Первый операнд стал в СМ, второй в Р)

y6: <РОП>(СМ, Р) → СМ;
(Первый операнд в СМ, второй в Р)

y7: оп →РОП; СМ→ Р;

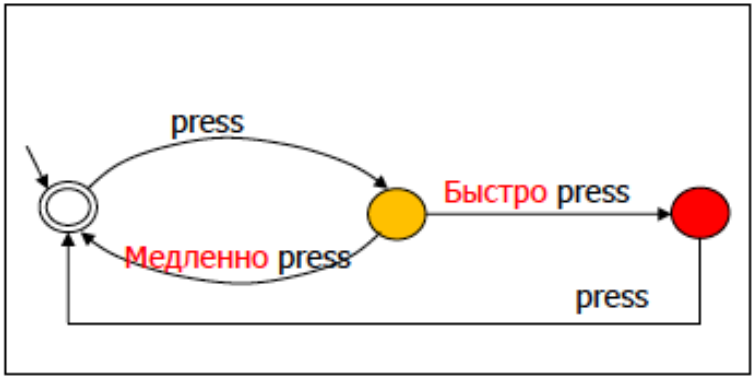
9 – 2 = = + = = 15

Временные конечные автоматы (Timed automata)



Интеллектуальный выключатель

- Двойное быстрое нажатие – яркий, однократное – бледный свет лампы. Следующее нажатие – погасить лампу



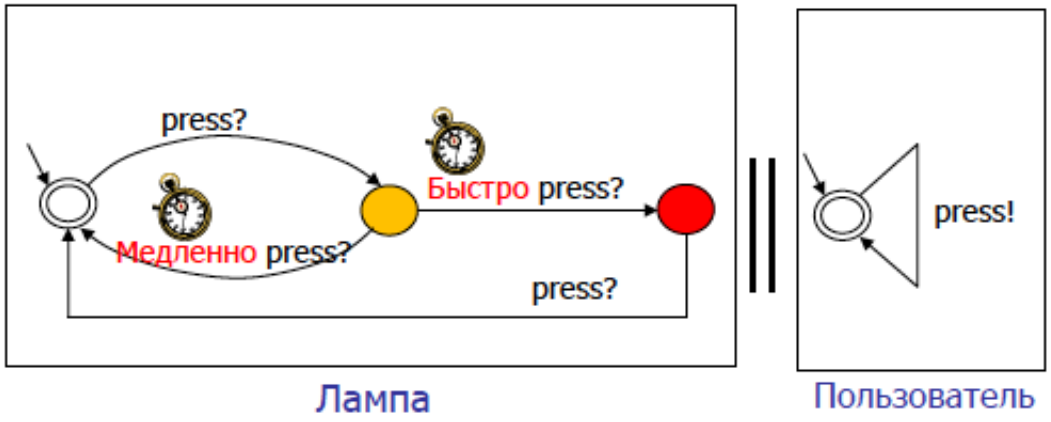
Система управления включением лампы - автомат



Какой формализм построить? Как расширить обычную модель?

Интеллектуальный выключатель

- Двойное быстрое нажатие – яркий, однократное – бледный свет лампы. Следующее нажатие – погасить лампу

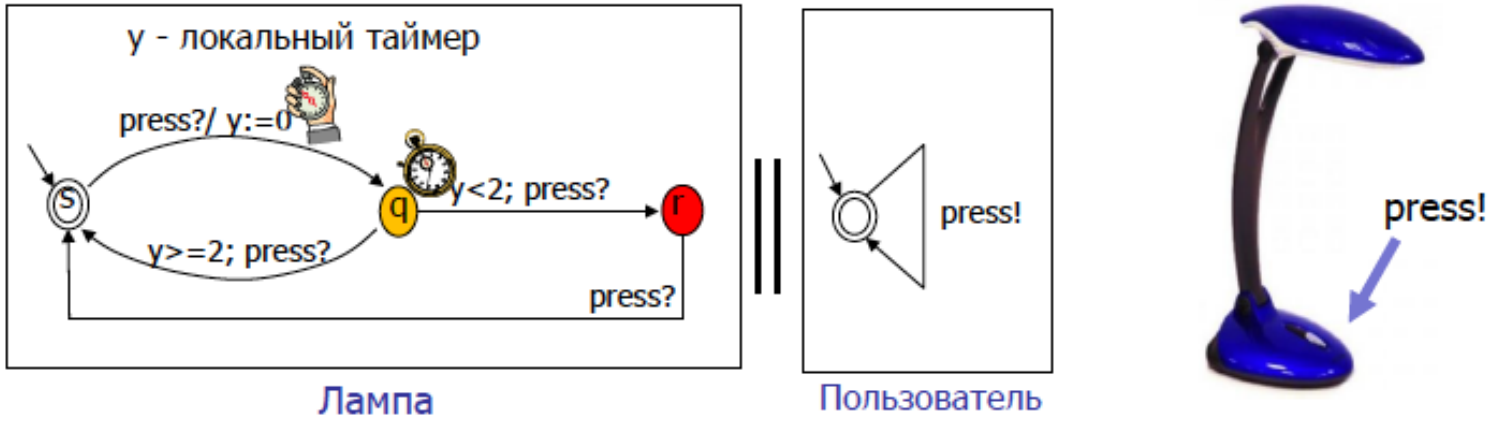




Временные автоматы (Timed Automata)

Необходим формализм, позволяющий конечным способом задать поведение, зависящее от реального времени

Вводилось много разных формализмов, чтобы адекватно представить поведение дискретных систем реального времени. Timed automata оказался наиболее удачным



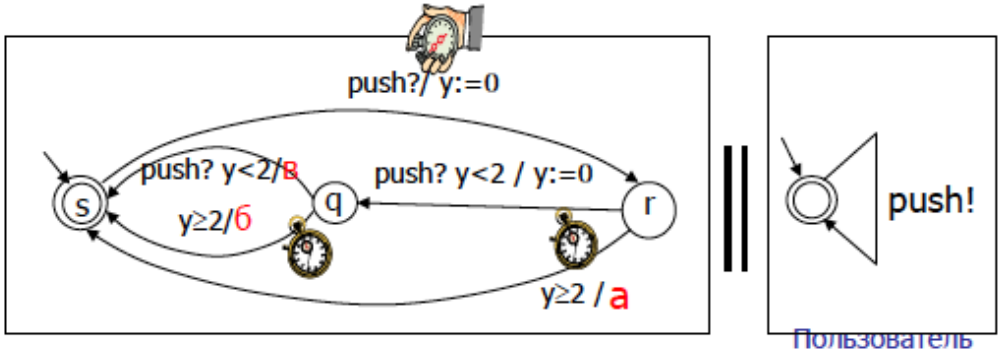
Решение: ввести локальный таймер и защиты переходов, являющиеся условиями над значениями локальных таймеров

Интеллектуальная кнопка мобильного телефона

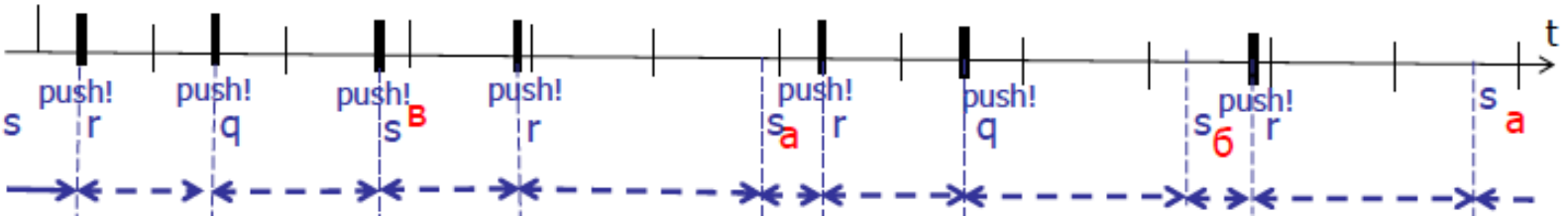
Как построить автомат с поведением:

Одно нажатие – а
Два медленных нажатия – аа
Двойное быстрое нажатие – б
Тройное быстрое нажатие – в





Задача: Что будет на выходе при следующей последовательности нажатий "push!"?



Конечные автоматы для представления программ и протоколов коммуникации



Программы как системы переходов

- State-transition systems встречаются в огромном числе областей. Состояние системы – это класс эквивалентных предысторий системы, однозначно определяющий ее дальнейшее поведение

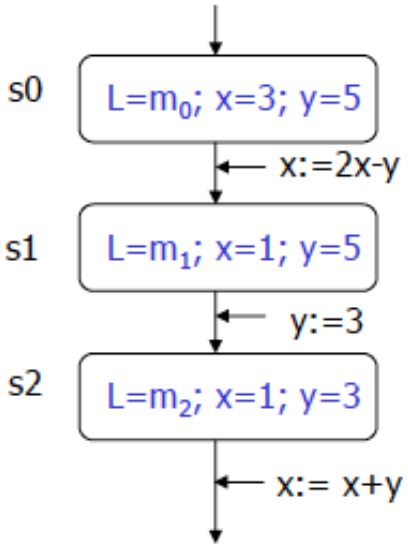
Pre={x=3; y=5;}

m_0: x := x-y;

m_1: y := 3;

m_2: x := x+y;

...



Состояние программы – это вектор значений всех переменных и счетчика команд

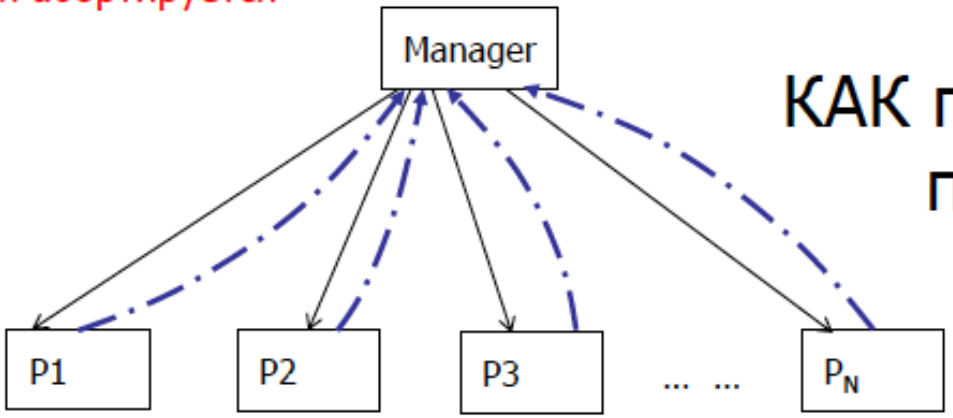
(В состояние включают состояние регистров и стека, если они используются программой)

Операторы программы меняют ее состояние

Если программа будет прервана после выполнения оператора (m_1: y:=3), то будет запомнено состояние программы: s2 = <L=m_2; x=1; y=3>

Протокол 2-х фазного распределенного соглашения (Two Phase Commit Protocol) обработки транзакций

Выработка соглашения в распределенной системе процессов: ЕСЛИ ВСЕ процессы соглашаются, все выполняют действие – обработку своей части транзакции. ЕСЛИ хотя бы один не соглашается, никто не выполняет действие, транзакция абортируется

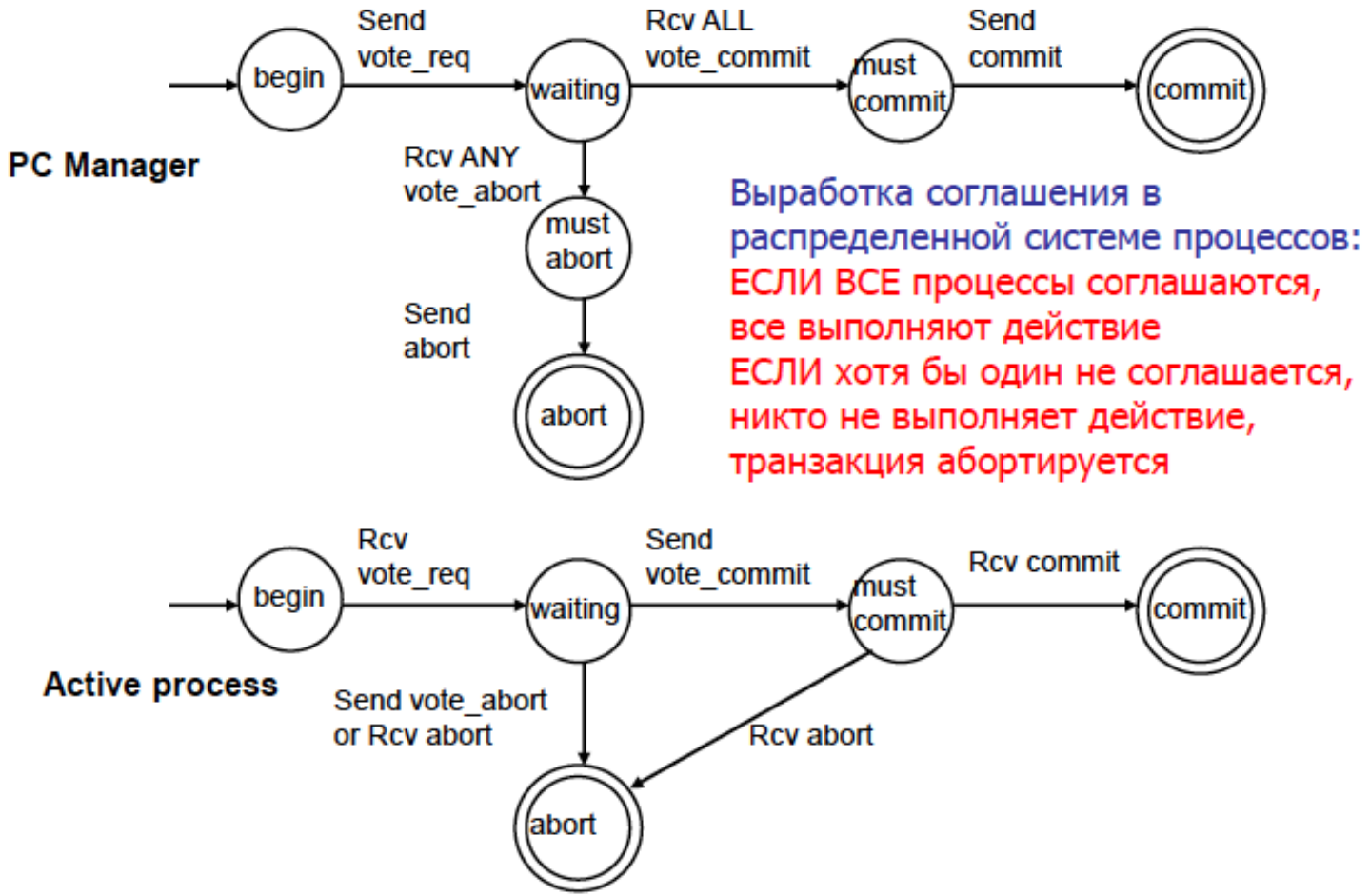


КАК построить протокол?

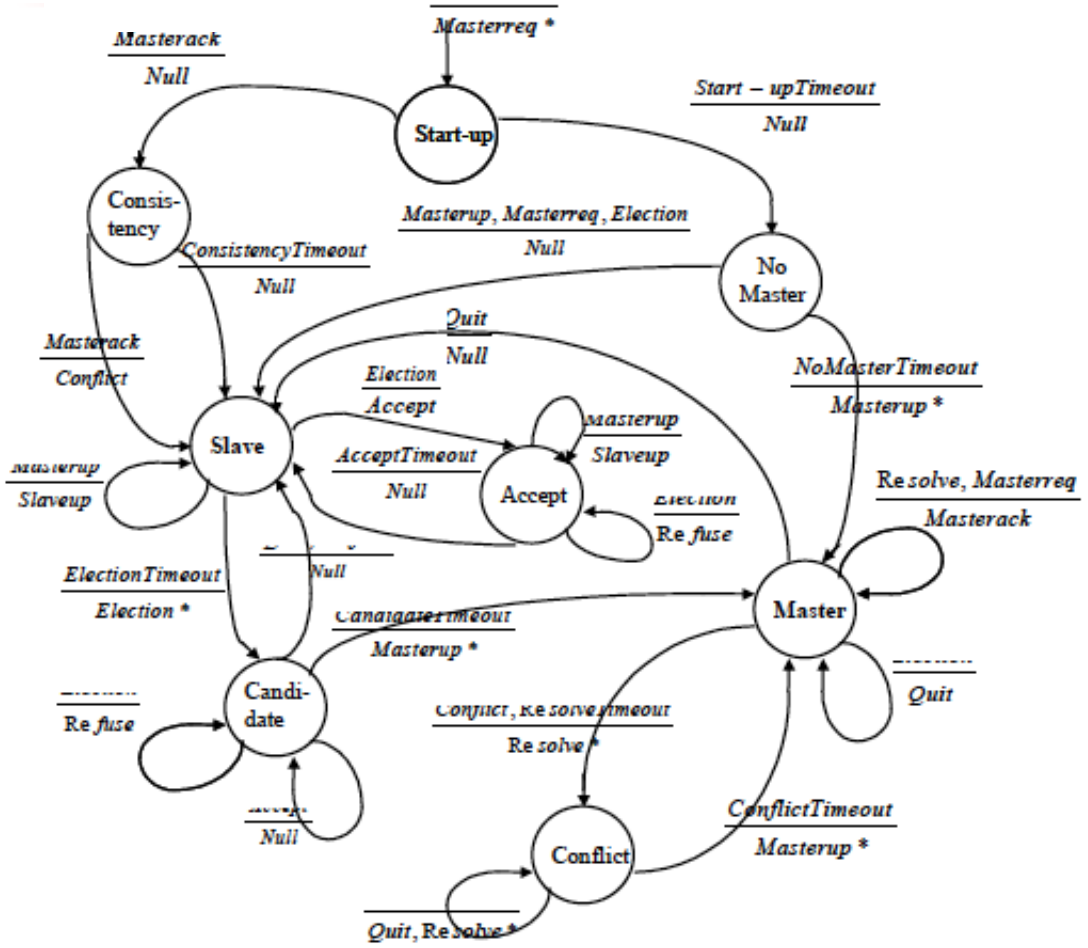
Первая фаза: бродкаст от процесса-менеджера запроса всем активным процессам и получение ответов от всех процессов

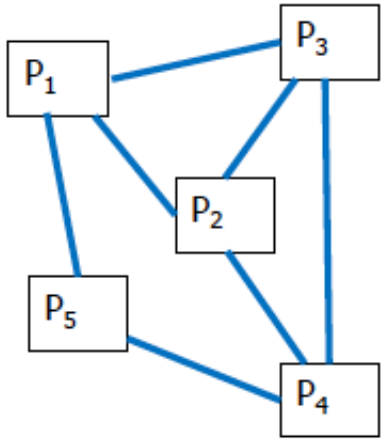
Вторая фаза:
процесс-менеджер посылает ABORT всем активным процессам, если хотя бы один активный процесс НЕ согласен,
процесс-менеджер посылает COMMIT всем активным процессам, если ВСЕ активные процессы согласны

Протокол 2-х фазного распределенного соглашения (Two Phase Commit Protocol) обработки транзакций



Протокол выбора лидера в сети как конечный автомат

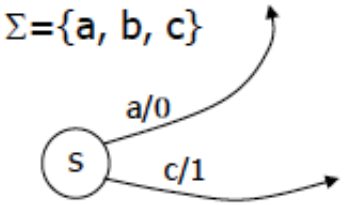




Протокол выбора лидера – идентичная программа в каждом компьютере, гарантирующая выбор единственного лидера, который знает, что он лидер, и все другие знают, что он лидер

Коллизии на переходах

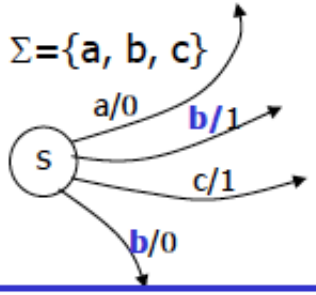
1. Не все возможные входные сигналы определены на переходах из состояния



Что делать, если придет b ?

- Три варианта семантики недоопределенности
  - Этот вход в реализации невозможен – доопределять, как угодно
  - Этот вход в данном состоянии является ошибкой – дополнить явным переходом в состояние ошибки
  - Этот вход нейтральный, не изменяющий состояние и с пустым выходом – явно определить этот переход в s (в себя)
- На практике все варианты возможны – НО конкретная семантика должна быть уточнена

2. Несколько переходов с одним и тем же входным сигналом



Что делать, если придет b ?

- Обычно в конечных автоматах-преобразователях и системах управления это свидетельствует об ошибке

Применение конечных автоматов в программировании

- Конечные автоматы применимы как модели алгоритмов для непосредственного программирования задач управления объектами.
- Основная проблема, которая возникает в случае прямого программирования в автоматной модели, - определение *множества состояний КА* и *входного алфавита*. Эта проблема легко решается в преобразовании блок-схемы программы в КА.
- **Утверждение**. Существует конструктивный метод преобразования блок-схем алгоритмов в КА.

- *Метод преобразования блок-схем в КА хорошо известен и апробирован, в основном, для дальнейших преобразований алгоритмов в схемы.*

- Модель алгоритма в виде КА легко преобразуется в блок-схему и, следовательно, всегда может быть выполнен переход к формальному описанию алгоритма на алгоритмическом языке и в виде программы ВМ.

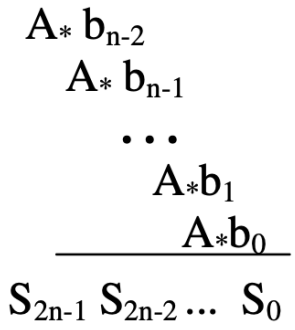
Применение метода

- В преобразованиях алгоритмов, представленных блок-схемами, в КА могут быть использованы **частичные КА**.
- Смысл существования частичных автоматов в программировании зависит от типа задачи, решаемой алгоритмическим методом:
  - для распознавателей языка – это ограничение на регулярный язык;
  - для алгоритмов преобразования данных – ограничение на область значений данных – результатов выполнения преобразований;
  - для алгоритмов управления – ограничение на входные данные.

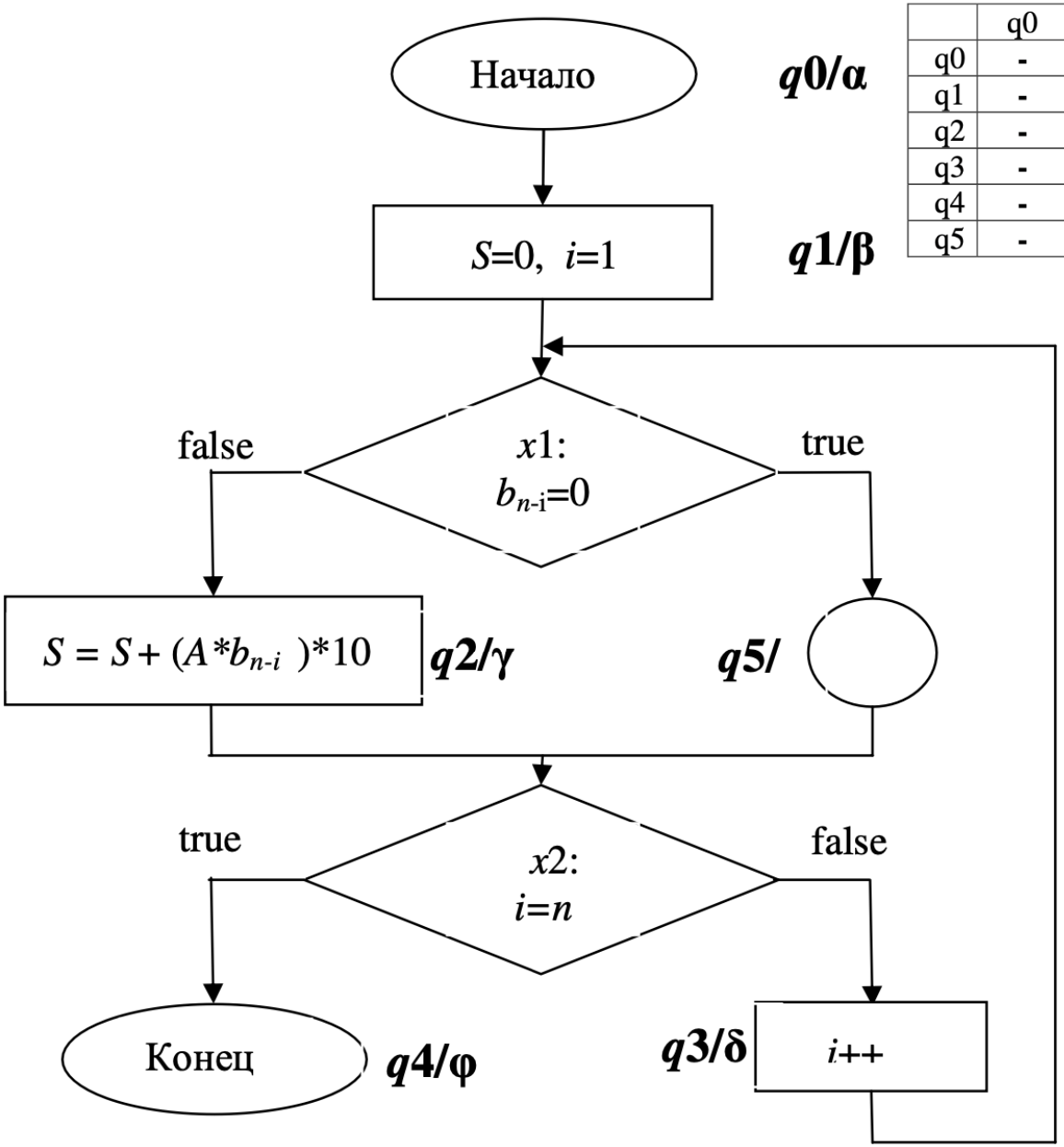
В случае распознавателей – признаки частичных автоматов могут быть использованы для контроля и выявления предложений, не принадлежащих языку при тестировании.

- В остальных случаях частичные автоматы также используются для **тестирования**.
- В реальных программах возможно ***доопределение переходов*** любым способом, так как соответствующие переходы в правильно работающей программе невозможны. Доопределение можно использовать для контроля и тестирования программы в процессе исполнения.

Пример: умножение чисел



- Рассмотрим преобразование блок-схемы алгоритма **умножения S=A*B**.
- Сомножители A = ($a_{n-1}a_{n-2}$ ... a_1a_0) и B = ($b_{n-1}b_{n-2}$ ..., b_1b_0) – n-разрядные десятичные числа, A - множимое, B - множитель.
- Операторные вершины, а также начальная (ввод) и конечная (вывод), обозначаются как состояния КА и имеют смысл задержки, необходимой для выполнения соответствующих операций. В алгоритме можно выполнять указанные операции или формировать команды для их автоматического исполнения в машине (вычислителе).
- Множество состояний Q = {q_0, q_1,..., q_5}.
- Входной алфавит определяет множество символов, кодирующих обозначенные в условных вершинах предикаты и принимающие двоичные значения {true, false}.
- Переходы из одного состояние в другое могут быть представлены конъюнкциями значений входных переменных {x_1, x_2}.
- Функция выходов, формируемые в состояниях, можно обозначить символами-командами выходного алфавита W = {α, β, δ, γ, φ} и поставить им в соответствие состояния КА.

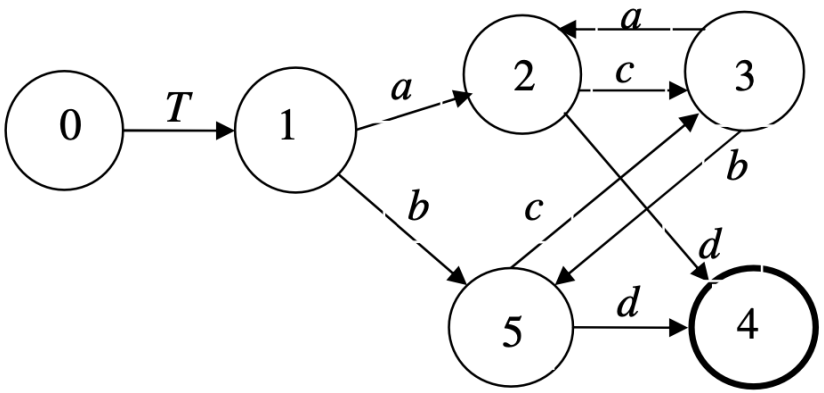


	q0	q1	q2	q3	q4	q5
q0	-	***T***	-	-	-	-
q1	-	-	$\bar{x}1$	-	-	***x*1**
q2	-	-	-	$\bar{x}2$	***x*2**	-
q3	-	-	$\bar{x}1$	-	-	***x*1**
q4	-	-	-	-	-	-
q5	-	-	-	$\bar{x}2$	***x*2**	-

Конечный автомат алгоритма умножения

Выполним замену логических условий символами входного алфавита $\Sigma^*=\{T, a, b, c, d\}$, где $a=\overline{x}1$, $b=x1$, $c=\overline{x}2$, $d=x2$, $q0$ - начальное состояние, $q4$ - финальное состояние.

	$q1$	$q2$	$q3$	$q4$	$q5$
$q0$	T	-	-	-	-
$q1$	-	a	-	-	b
$q2$	-	-	c	d	-
$q3$	-	a	-	-	b
$q4$	-	-	-	-	-
$q5$	-	-	c	d	-



Конечный автомат алгоритма умножения

$$L(M) = (0T1)\{[(1a2)(((2c3)(3a2))^*(2d4)+(2c3)((3b5)(5c3))^*(3b5)(5d4)\ ] +$$

$$+(1b5)[((5c3)(3b5))^*(5d4)+(5c3)((3b5)(5c3))^*((3a2)(2c3))^*(3a2)(2d4)]\}=$$

$$=T[a((ca)^*d+c(bc)^*bd)+b((cb)^*d+c(bc)^*(ac)^*ad]=$$

$$=T[a((ca)^*+c(bc)^*b)+b((cb)^*+c(bc)^*(ac)^*a]d=$$

$$=T[a((ca)^* +c(bc)^*b) +b((cb)^* +c(bc)^*(ac)^*a]d=$$

$$= T[a((ca)^* +tb) +b((cb)^* +t(ac)^*a]d,$$

t - переводит КА в состояние, которое запускает КА, распознающий строку $c(bc)^*$.

История: 60 лет использования модели КА

- Модель конечного автомата используется уже 60 лет - с 50-х годов 20 века:
  Huffman, D. A., "The synthesis of sequential switching circuits," *Journal of the Franklin Institute,* 257 (1954)

- Это одна из старейших моделей, использованных для описания системы преобразования информации, выход которой зависит не только от входа, но и от истории:



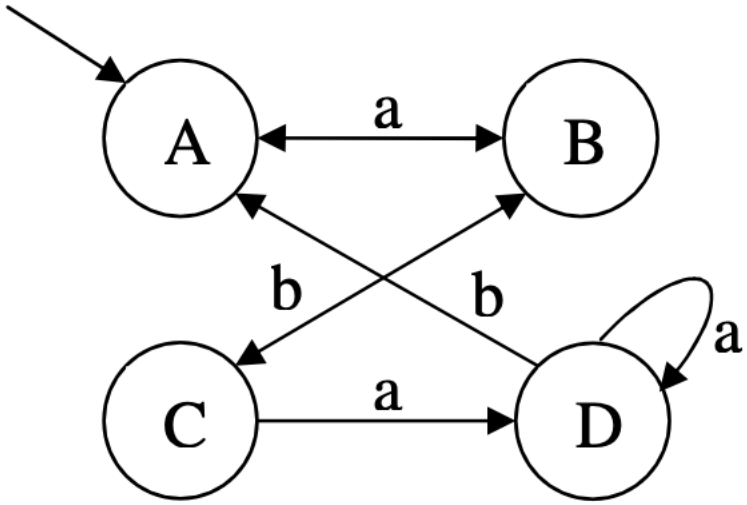
Заключение

- Одно из приложений двоичных функций – реализация конечных функциональных преобразователей и конечных автоматов
- **Конечные автоматы – формальная математическая модель. Теория конечных автоматов – это набор концепций, теорем и утверждений о свойствах этой модели**
- КА – это формальная модель преобразователей информации с конечной памятью. Выход КА зависит не только от входа, но и от предыстории, т.е. от цепочки входных сигналов, которые были поданы раньше, с момента начала функционирования автомата
- Состояние КА – это класс эквивалентных предысторий
- В конечном автомате конечное число эквивалентных классов всех историй (множеств эквивалентных - с точки зрения дальнейшего функционирования автомата - цепочек входных символов)
- **С помощью модели конечного автомата могут представляться преобразователи информации, алгоритмы систем логического управления, протоколы, программы**
- По формальной модели КА может быть построено реальное устройство преобразования информации: конечный автомат может быть реализован и программно, и аппаратно многими разными способами

СПАСИБО
ЗА ВНИМАНИЕ

Пример: таблица переходов и КА

Qi	Σ	Qj
A	a	B
	b	--
B	a	A
	b	C
C	a	D
	b	B
D	a	D
	b	A



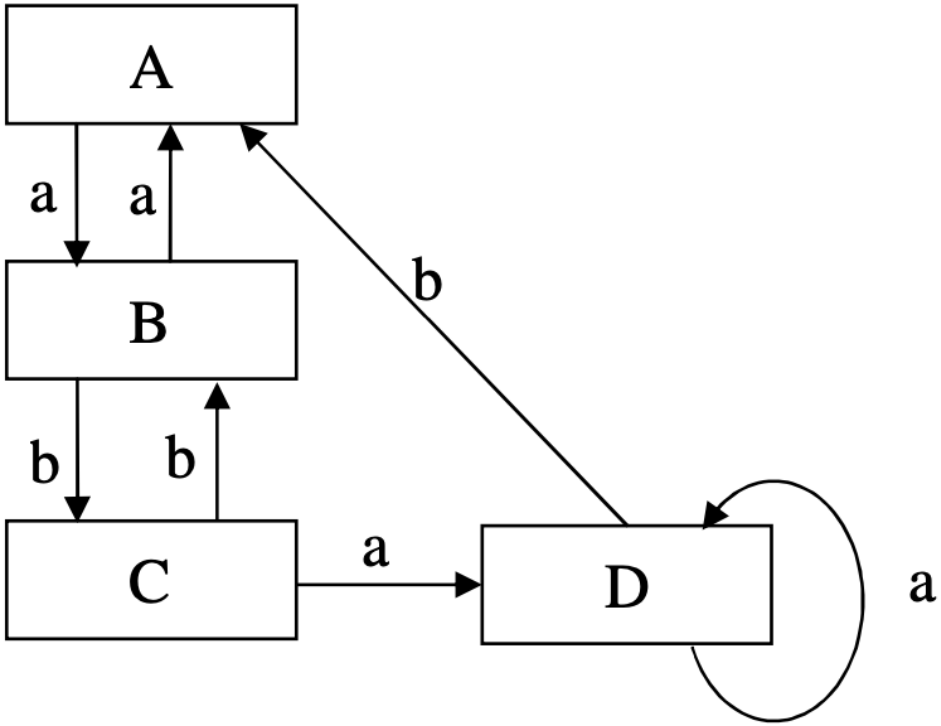
- Регулярное выражение.

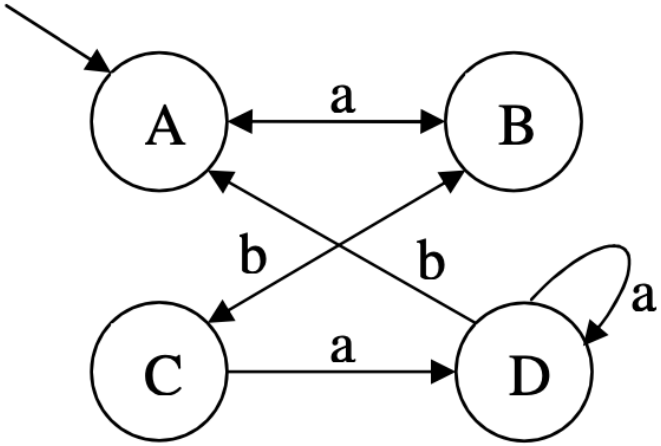
S=(((A aB)(BaA))*(AaB)((BbC)(CbB))*(BbC)(CaD)(DaD)*(DbA))*

= ((a a)* a (b b)* b a a* b)* = ((a a)* a(bb)* baa*b)*

A B A B C B C D D A

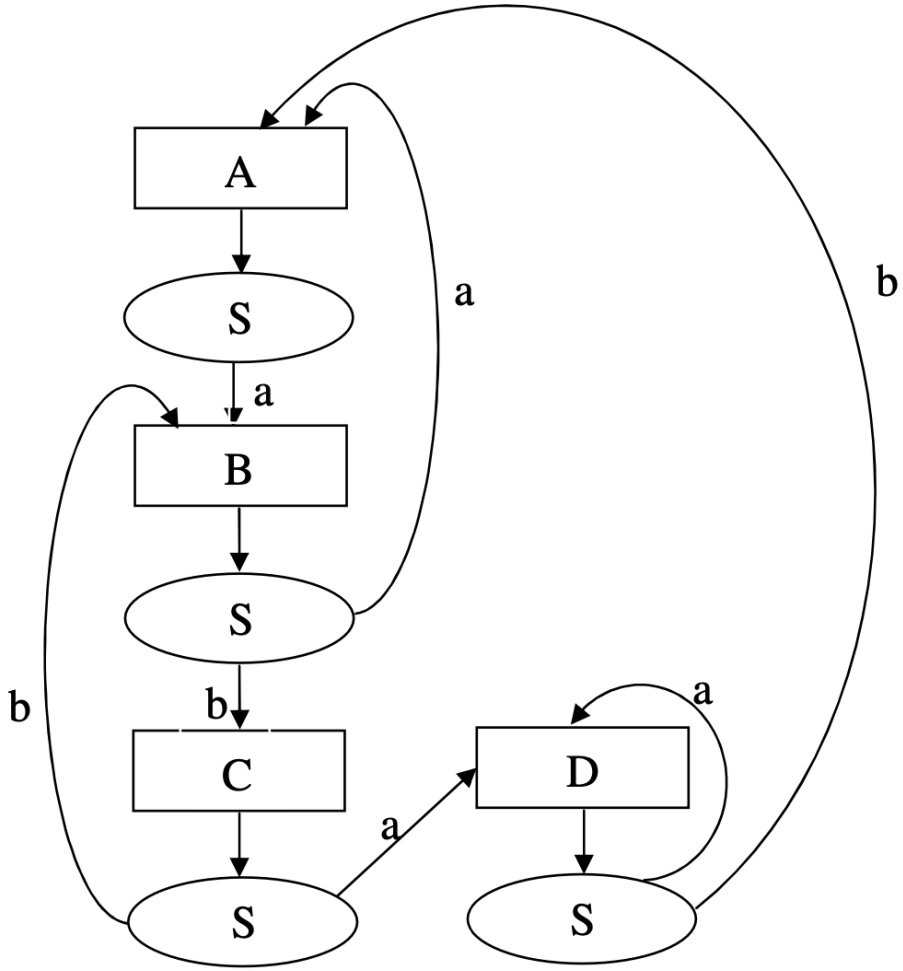
Пример: КА и блок-схема





Состояния представлены операторной вершиной.

Пример (продолжение)



Ребра графа представлены переключателем (switch).

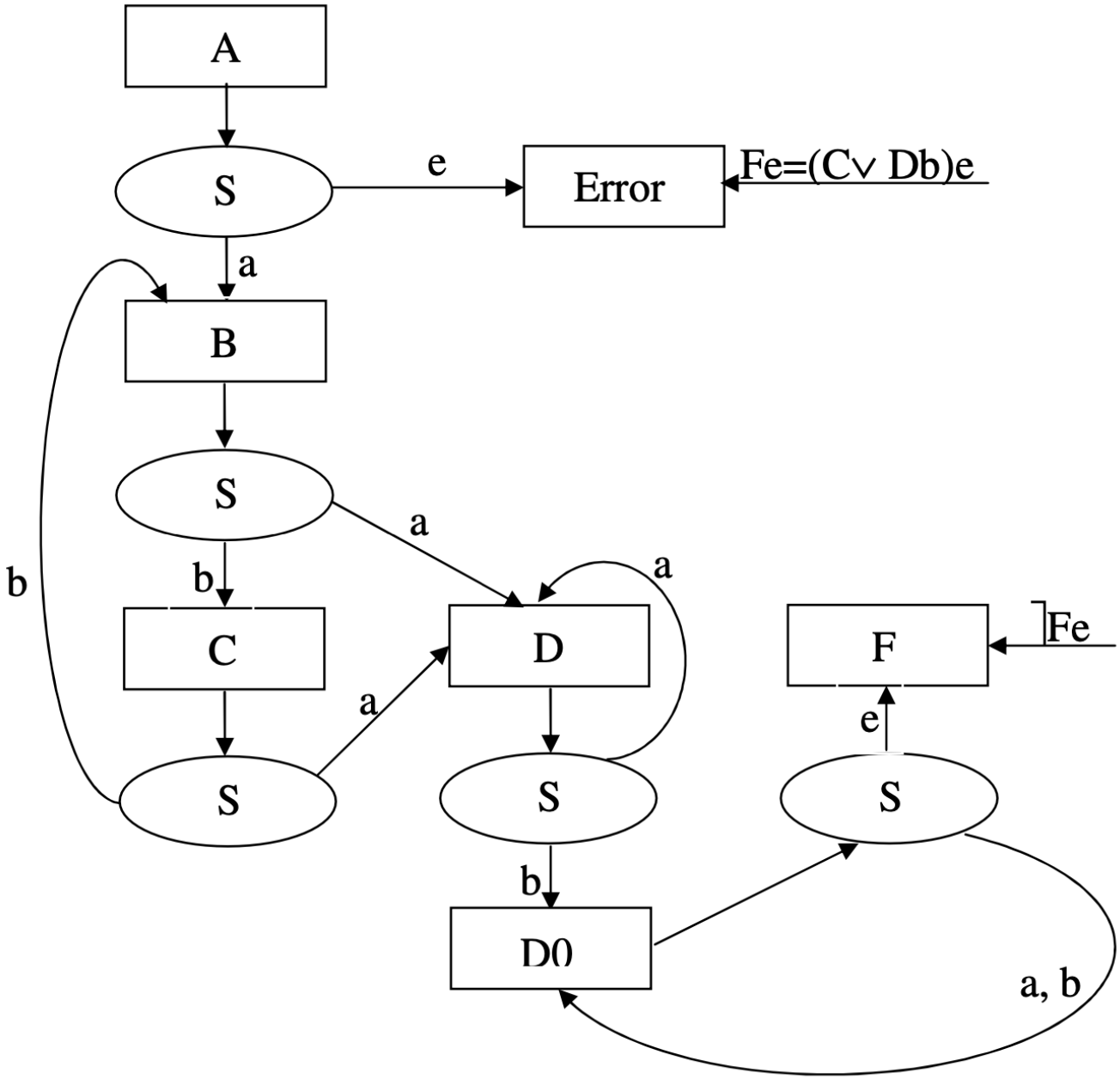
- Переход безусловный, если в условии присутствуют все символы входного алфавита.
- В каждом состоянии выполняется некоторая команда из алфавита {A, B, C, D}.
- Расширяем входной алфавит пустым символом {e}, множество состояний – символом {F}, обозначающим финальное состояние (успешное завершение идентификации слова, принадлежащего языку L(M)).
- Если условие перехода по символу не определено (не полностью определенный КА), то добавляется состояние E, контролирующее ошибку.
- В конце непустой цепочки символов всегда (be). В примере это условия Ce или Dbe, все другие условия – ошибочные.

Пример

- Можно определить правильное завершение распознавания и, соответственно, контролировать логику выполнения алгоритма уравнением
  - **Fe= Ce ∨ Dbe=(C ∨ Db)e.**

- При ошибке в цепочке Ee= ¬Fe.
- Для вычисления Fe можно добавить состояния (Fe) и операторную вершину в блок-схеме, что подразумевает недетерминированность при бесконечно больших цепочках.
- Команда (Fe) завершает алгоритм при пустой цепочке.

- *Алгоритм можно использовать для доопределения программы и автоматического тестирования логики программы.*


A
S
e
Error
Fe=(C∨ Db)e
a
B
S
a
b
b
a
C
D
F
Fe
a
e
S
S
S
b
D0
a, b